

MAJOR PROJECT REPORT
On
Food Nutrition Estimator from Image and Text using AI

Submitted to
Amity University, Uttar Pradesh



In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

Under the guidance of:

Dr Roshan Lal

By:
Aryan Walia(A023166922038)
Subham Prasad Archary(A023166922045)
Kushagra Pankaj(A023166922048)

B. TECH CSE IOT AND CYBERSECURITY INC BCT

2022-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY

AMITY UNIVERSITY UTTAR PRADESH

NOIDA (U.P.)

DECLARATION

We, Aryan Walia, Subham Prasad Archary, Kushagra Pankaj student of B. Tech CSE (IOT and Cybersecurity including BCT) hereby declare that the Major Project Report titled “Food Nutrition Estimator From Image and Text using AI”, which is submitted by us to the Department of Computer Science and Engineering, Amity School of Engineering and Technology, Amity University Uttar Pradesh, Noida, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition.

Place: NOIDA

Aryan Walia
A023166922038

Subham Prasad
Archary
A023166922045

Kushagra
Pankaj
A023166922048

CERTIFICATE

On the basis of the declaration submitted by ARYAN WALIA, SUBHAM PRASAD ARCHARY, KUSHAGRA PANKAJ student of B.TECH Computer Science and Engineering (Internet Of Things and Cybersecurity including Block Chain Technology), I hereby certify that the Minor Project titled “Food Nutrition Estimator From Image and Text using AI” which is submitted to Department of Computer Science and Engineering, Amity School of Engineering and Technology, Amity University, Noida, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is an original contribution with existing knowledge and faithful record of work carried out by him/her under my guidance supervision.

To the best of my knowledge, this work has not been submitted in any part or full for any degree or diploma to this university or somewhere else.

Place: NOIDA

Dr Roshan Lal

Associate Professor Grade-III
Department of Computer Science and Technology
Amity School of Engineering and
Technology
Amity University Uttar Pradesh,
Noida, UP

DECLARATION FORM

I, Aryan Walia, student of B. Tech Computer Science & Engineering, Enrollment No. A023166922038, batch 2022-2026, Department of Computer Science & Engineering, Amity School of Engineering and Technology, Domain of Amity University Uttar Pradesh, Noida hereby declare that I have gone through project guidelines including policy on health and safety, policy on plagiarism etc.

Place: Noida

Date:

Student Signature

DECLARATION FORM

I, Subham Prasad Achary, student of B. Tech Computer Science & Engineering, Enrollment No. A023166922045, batch 2022-2026, Department of Computer Science & Engineering, Amity School of Engineering and Technology, Domain of Amity University Uttar Pradesh, Noida hereby declare that I have gone through project guidelines including policy on health and safety, policy on plagiarism etc.

Place: Noida

Date:

Student Signature

DECLARATION FORM

I, Kushagra Pankaj, student of B. Tech Computer Science & Engineering, Enrollment No. A023166922048, batch 2022-2026, Department of Computer Science & Engineering, Amity School of Engineering and Technology, Domain of Amity University Uttar Pradesh, Noida hereby declare that I have gone through project guidelines including policy on health and safety, policy on plagiarism etc.

Place: Noida

Date:

Student Signature

ACKNOWLEDGEMENT

I take this opportunity to express my profound sense of gratitude and respect to all those who helped me throughout my project. This report acknowledges the intense driving and technical competence of everyone who has contributed to it. It would have been almost impossible to complete this project without the support of these people.

I am grateful to my faculty guide Dr. Roshan Lal, Assistant Professor - III, Amity School of Engineering and Technology (ASET) for guiding me and helping me out with the slightest of details in completing this project in the best way possible. I extend thanks and gratitude to him to have imparted me his valuable guidance in all aspects.

I am also grateful to Prof. (Dr.) Sanjeev Thakur, (HoD), Department of Computer Science & Engineering, for their regular encouragement and motivation.

Also, I would like to extend my gratitude to my family and friends who helped me and encouraged me to complete this project with utmost sincerity and hard work.

Without their encouragement and help, this work would have been an ordeal for me to tackle. Above all, I pay my reverence to the almighty GOD.

Place: Noida

Date:

Aryan Walia (A023166922038)

Subham Prasad Achary (A023166922045)

Kushagra Pankaj (A023166922048)

Abstract

LiveFit is a comprehensive, full-stack AI-powered nutrition and meal management platform designed to bridge the gap between food awareness and personalised dietary guidance. In an era where non-communicable diseases linked to poor nutrition are on a dramatic rise, especially across urban India, users increasingly require intelligent, accessible, and personalised tools to manage their dietary intake. LiveFit addresses this need by seamlessly combining a cross-platform mobile application (built with React Native and Expo), a modern web interface, and a robust cloud-hosted backend into a single cohesive ecosystem. The core of the platform is an intelligent food identification engine that enables users to log meals either by typing the food name or uploading a photograph. When an image is submitted, a hybrid machine-learning pipeline is invoked: a locally-hosted Python FastAPI microservice runs the fine-tuned Hugging Face transformer model 'rajistics/finetuned-indian-food', specifically optimised for recognising Indian dishes. If the model's confidence falls below 60%, the system transparently falls back to the Roboflow computer vision API, ensuring robust identification even for unusual or ambiguous food photographs. The identified food name is then resolved against a prioritised, dual-database nutritional lookup strategy. For Indian cuisine, the system queries a locally embedded Indian Food Composition Tables (IFCT) dataset using exact, partial, and reverse fuzzy matching, providing highly accurate macronutrient data for thousands of subcontinental foods. For all other food items, the system queries the USDA FoodData Central API, a globally authoritative nutritional database.

Beyond food logging, LiveFit integrates Google Gemini 2.5 Flash as a conversational AI nutrition coach. This AI assistant is provided with the user's real-time nutritional intake data and responds to natural language questions with personalised, actionable dietary advice, identifying macro deficiencies or excesses and suggesting corrective food choices. The platform further features a gamified reward ecosystem wherein users earn LiveFit Coins for consistent food logging, which can then be redeemed in a dedicated store for tangible benefits including percentage-based discount codes, free meals, Pro subscription trials, and nutrition reports. The discount codes integrate directly with the Razorpay payment gateway, enabling users to apply validated coupon codes at checkout with real-time price adjustment. A subscription tier system (Free and Pro) unlocks advanced features including unlimited AI coaching interactions and enhanced analytics. The meal ordering module allows users to browse a curated selection of health-optimised meals, add them to a persistent cart, and complete purchases through Razorpay, with PDF invoices automatically generated upon order placement. All user data is securely stored in a cloud MongoDB Atlas database, with authentication handled via bcrypt password hashing and JSON Web Tokens enforcing stateless session management throughout the application.

Table of Contents

Section	Title	Page No.
Chapter 1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Proposed Solution	3
1.4	Objectives	4
1.5	Scope of the Project	6
1.6	Report Organisation	7
1.7	Definitions and Acronyms	7
1.8	Motivation and Personal Relevance	8
1.9	Innovation Contributions	8
Chapter 2	Literature Review	9
2.1	Nutrition Applications — State of the Art	9
2.2	AI/ML in Food Identification	10
2.3	Gamification in Health Platforms	11
2.4	Conversational AI in Nutrition Coaching	12
2.5	Challenges and Research Gaps	14
2.6	Synthesis and Positioning of LiveFit	14
2.7	Research Gaps Addressed	15
Chapter 3	Technologies & Tools	16
Chapter 4	System Design & Architecture	24
Chapter 5	Implementation	33
Chapter 6	Security, Privacy and Scalability	48

Chapter 7	Limitations	52
Chapter 8	Results	54
Chapter 9	Conclusion	69
—	References	70

List of Figures

Figure No.	Title	Page No.
Fig. 1.1	High-Level System Architecture	6
Fig. 3.1	Food Identification Pipeline — Hybrid Classification Flow	20
Fig. 3.2	AI Nutrition Coaching Data Flow	21
Fig. 4.1	MongoDB Data Model — Collection Relationships	27
Fig. 4.2	Gamification and Reward System Architecture	30
Fig. 5.1	Payment and Order Processing Flow	39
Fig. 5.2	React Native Mobile Application — Screen Overview	44
Fig. 6.1	Security Architecture — Eight Pillars	50
Fig. 8.1	API Performance Metrics: Average and 95th-percentile Latency	55
Fig. 8.2	Error Rate (%) by Endpoint Category	56
Fig. 8.3	LiveFit Website Landing Page	56
Fig. 8.4	Sign Up Page (left) Login Page (right)	57
Fig. 8.5	Meal Tracker (empty state)	57
Fig. 8.6	Meal Tracker (filled state)	57
Fig. 8.7	Text Food Input and Image Food Input	58

Fig. 8.8	Total Calorie Count Dashboard — Daily Summary	58
Fig. 8.9	Personalised Nutrition Graph and Suggestions	59
Fig. 8.10	Meal Ordering System — Healthy Meals Catalogue	59
Fig. 8.11	Checkout Cart — Line Items, Reward Code Input, Order Summary	60
Fig. 8.12	Rewards Store — Coin Balance, Available and Locked Rewards	60
Fig. 8.13	Subscription Plans — Free, Pro, and Elite Tiers	61
Fig. 8.14	Razorpay Payment Modal — UPI QR for Order	61
Fig. 8.15	MongoDB Atlas Collections — All 7 LiveFit Collections	62
Fig. 8.16	AI Personalised Chatbot — Gemini 2.5 Flash Response	62
Fig. 8.17	IFCT vs. USDA: Coverage, Accuracy, and Error Comparison	64
Fig. 8.18	AI Coaching Quality: Scores Across Five Evaluation Criteria	65
Fig. 8.19	Coin Balance Progression: Earning and Redemption	66
Fig. 8.20	Load Testing: Response Time and Timeout Rate vs. Concurrent Users	67

List of Tables

Table No.	Title	Page No.
Table 1.3.1	Core Components of LiveFit	4
Table 3.7.1	LiveFit Software Bill of Materials	22
Table 4.5.1	LiveFit REST API Endpoints	27
Table 4.5.2	LiveFit Coins Rewards	30
Table 5.1	Pseudocode: Signup	33
Table 5.2	Pseudocode: Login	34
Table 5.3	Pseudocode: Image Processing	34
Table 5.4	Pseudocode: Local Classifier Model	35
Table 5.5	Pseudocode: Database Lookup	35
Table 5.6	Pseudocode: Food Logging	36
Table 5.7	Pseudocode: AI Chatbot	37
Table 5.8	Pseudocode: Coins System	37
Table 5.9	Pseudocode: Payment Module	38
Table 5.10	Pseudocode: Cart Checkout	39
Table 5.11	Pseudocode: Request Reset	40
Table 5.11.1	Pseudocode: Confirm Reset	40
Table 5.12	Pseudocode: Nutrition Calculation	41
Table 5.13	Pseudocode: Session Storage	41
Table 5.14	Pseudocode: Add to Cart	42
Table 5.15	Pseudocode: Coupons Applied	42
Table 5.16	Pseudocode: Subscription Module	42
Table 8.1	API Performance Benchmark Results	54

Table 8.2	Error Rate (%) by Endpoint Category	55
Table 8.3	Food Identification: Text Input Test Cases	63
Table 8.4	Image Classification Pipeline Test Cases	63
Table 8.5	IFCT vs. USDA Accuracy Comparison for 30 Indian Dishes	64
Table 8.6	AI Coaching Quality Assessment (20 Test Scenarios)	65
Table 8.7	Reward System End-to-End Test Results	65
Table 8.8	Payment Flow Validation Results	66
Table 8.9	Load Testing Results	67
Table 8.10	Usability Testing Results	67
Table 8.11	Security Testing Results	68
Table 8.12	Consolidated Results Summary	69

Chapter 1: Introduction

1.1 Background

The relationship between diet and health has never been better understood scientifically, yet translating nutritional knowledge into daily practice remains one of the most persistent public health challenges of the modern era. According to the World Health Organisation, an estimated 2.8 million people die annually as a result of being overweight or obese, with a further enormous burden attributable to micronutrient deficiencies and diet-related non-communicable diseases including diabetes, cardiovascular disease, and certain cancers [27]. In India specifically, the dual burden of undernutrition and the rapid rise of obesity and metabolic syndrome create a uniquely complex nutritional landscape that demands culturally sensitive, technologically advanced solutions.

Traditional dietary tracking methods, including paper food diaries, generic calorie counting tables, and one-size-fits-all nutritional advice, have proven systematically inadequate. They are burdensome, imprecise, culturally ill-adapted to the extraordinary variety of Indian cuisine, and fundamentally disconnected from the real-time feedback loops that modern behavioural science has shown to be essential for sustainable habit formation. The proliferation of smartphones, especially among India's rapidly expanding middle class, presents a historic opportunity to deploy intelligent, personalised nutrition tools at population scale.

LiveFit was conceived to fill precisely this gap [4][5]. Rather than offering a static calorie counter, LiveFit provides an active nutritional intelligence system that meets users where they are, whether photographing a home-cooked thali or searching for the macronutrients in a packaged food product, and responds with immediate, contextualised, and personalised dietary guidance. The platform is designed from the ground up to be Indian-first, incorporating the IFCT nutritional database and a fine-tuned Indian food recognition model, while remaining fully capable of handling global cuisines through USDA integration.

1.2 Problem Statement

Despite the proliferation of health and fitness applications in recent years, a significant segment of users, particularly in India, continue to face substantial barriers when attempting to accurately track their nutritional intake. These barriers manifest across several interconnected dimensions.

First, there is the problem of Indian food coverage. The vast majority of existing nutrition applications are built on Western food databases, particularly the USDA database, which has very limited coverage of the thousands of distinct Indian dishes consumed daily across the subcontinent. A user attempting to log 'dal makhani', 'aloo

paratha', or 'rasam' will frequently find either no match, an incorrect match, or nutritional data that bears little relationship to the actual composition of the dish as typically prepared in India.

Second, there is the friction of manual food logging. Research consistently demonstrates that the single greatest predictor of dietary tracking adherence is the cognitive and temporal burden of the logging process itself [4][6]. Requiring users to manually search for and select foods, then estimate portion sizes and manually enter quantities, creates sufficient friction to cause widespread abandonment of tracking applications within days of installation.

Third, there is the absence of intelligent, personalised feedback. Most nutrition applications function essentially as passive ledgers, recording what has been consumed without providing actionable guidance on what adjustments would best serve the user's health goals. Without this intelligent feedback loop, the motivational value of tracking data diminishes rapidly.

Fourth, there is the challenge of sustaining user engagement over time. Health behaviour change is inherently a long-term project, and applications that fail to maintain user engagement beyond the initial novelty phase cannot deliver meaningful health outcomes. Gamification, social features, and reward mechanisms have been shown in the literature to significantly improve long-term engagement, but few nutrition applications implement these mechanisms in a sophisticated or genuinely rewarding way.

LiveFit addresses all four of these problems through an integrated system architecture that prioritises Indian food recognition, minimises logging friction through AI-powered identification, provides real-time intelligent coaching through generative AI, and sustains engagement through a meaningful coin and reward ecosystem.

1.2.1 The Indian Nutrition Gap in Detail

India's nutritional diversity is staggering. With over 7,500 distinct dishes documented across 28 states and 8 union territories, each with unique ingredients, preparation methods, and regional variants, no single Western-designed database can adequately serve an Indian user. Consider the simple case of 'sambar': the Tamil Nadu version differs markedly in composition from the Karnataka or Kerala variant, and the home-cooked version differs further still from restaurant preparations. Caloric estimates derived from generic databases may err by 30–50% for such dishes, systematically corrupting the dietary records of millions of users who rely on these applications for health management.

The problem is compounded by the structure of traditional Indian meals, which rarely conform to the discrete, individually identifiable food items that database-centric applications expect. A typical North Indian thali contains six to ten components

simultaneously, many of which are preparation-specific combinations for which no database entry exists. South Indian tiffin, Bengali fish curries, and Gujarati farsan all present similar challenges. This structural mismatch between the Indian dietary pattern and the assumptions embedded in existing digital tracking tools is the single largest barrier to the adoption of nutrition applications across the Indian market.

1.2.2 The Engagement Deficit in Health Applications

Beyond accuracy, the health application industry faces a well-documented engagement crisis. Industry data consistently shows that over 80% of health and fitness application users abandon their chosen application within the first month of installation. The causes are multifactorial, including the friction of manual data entry, the absence of meaningful progress feedback, the lack of social accountability mechanisms, and the failure to provide tangible rewards for sustained engagement. Applications that succeed in maintaining long-term engagement consistently share common features: low-friction interaction design, intelligent personalisation, meaningful progression systems, and tangible incentive structures. LiveFit is designed from the outset to incorporate all four of these retention mechanisms.

The economic dimension of this engagement deficit is also significant. Users who abandon nutrition applications typically do so because the perceived effort cost exceeds the perceived value delivered. The asymmetry is particularly acute in the first two to four weeks of use, before any meaningful health outcomes become visible. Bridging this gap requires delivering immediate, tangible value at every interaction — a principle that directly motivates LiveFit's coin reward for each food item logged, ensuring that every tracking action delivers an immediate, visible, and economically meaningful return.

1.3 Proposed Solution

LiveFit proposes a comprehensive hybrid platform that brings together computer vision, natural language AI, curated nutritional databases, and gamified reward systems into a unified user experience. The solution operates across two complementary interfaces: a mobile application built with React Native and Expo for on-the-go food logging and coaching, and a web application built with vanilla HTML, CSS, and JavaScript for richer browsing and management experiences.

The food identification pipeline is the technical centrepiece of the system. When a user submits a food photograph, it is transmitted to a locally-hosted Python FastAPI microservice that runs a fine-tuned vision transformer model specifically trained on Indian food categories. This local model provides immediate, low-latency responses without incurring per-query API costs for the most common use case. A confidence threshold mechanism ensures that only high-confidence predictions are accepted from the local model; lower-confidence predictions trigger an automatic fallback to the

Roboflow computer vision API, which provides a robust safety net for unusual foods or photographs of suboptimal quality.

The nutritional resolution layer implements a prioritised dual-database strategy. Identified Indian food items are first looked up in the embedded IFCT dataset, which contains detailed macronutrient profiles for traditional Indian foods. Non-Indian foods, or Indian foods not found in IFCT, are resolved against the USDA FoodData Central API. This layered approach maximises accuracy for Indian cuisine while ensuring comprehensive coverage of global foods.

Table 1.3.1: Core Components of LiveFit

Feature	Implementation	User Benefit
Food ID by Image	HuggingFace + Roboflow hybrid ML pipeline	Zero-friction photo-based logging
Food ID by Text	IFCT + USDA dual-database lookup with fuzzy matching	High accuracy for Indian & global foods
AI Coaching	Google Gemini 2.5 Flash with real-time nutrition context	Personalised dietary advice 24/7
Gamification	Coin earning + redemption reward store	Sustained long-term engagement
Meal Ordering	Curated health-focused menu + Razorpay payment	Healthy food delivery integration
Subscriptions	Free / Pro tiers via Razorpay	Monetisation with premium features
Secure Auth	bcrypt + JWT + email reset	Enterprise-grade account security

1.4 Objectives

The project pursues the following primary objectives, each addressing a specific gap in the current landscape of nutrition technology applications.

Develop an end-to-end functional nutrition tracking platform supporting both mobile and web interfaces, enabling users across device types to access the full feature set.

Implement a high-accuracy hybrid food identification pipeline combining local fine-tuned machine learning with cloud-based computer vision fallback, achieving reliable food recognition across Indian and global cuisines.

Integrate a dual-database nutritional resolution system leveraging the IFCT dataset for Indian foods and the USDA FoodData Central API for global foods, providing accurate macronutrient data for thousands of food items.

Deploy a conversational AI nutrition coaching system powered by Google Gemini 2.5 Flash, capable of providing personalised dietary advice based on the user's real-time nutritional intake data.

Design and implement a gamified coin-and-reward ecosystem that incentivises consistent food logging and healthy meal ordering, improving long-term user engagement and platform retention.

Integrate a complete payment infrastructure supporting meal purchases and subscription upgrades via Razorpay, with automatic PDF invoice generation for all confirmed orders.

Ensure robust security throughout the platform through bcrypt password hashing, JWT-based stateless authentication, and secure API communication.

1.4.1 Technical Objectives

From a technical standpoint, the project aims to demonstrate that a production-quality AI-powered health platform can be built and deployed using entirely open-source tools and free-tier cloud services, making the architectural approach replicable and accessible for future research and commercial development. The system specifically targets the following technical benchmarks: food identification latency below 2 seconds for image-based queries, nutritional lookup accuracy exceeding 90% for common Indian dishes, AI coaching response quality that matches or exceeds rule-based dietary advice systems, and payment transaction success rates matching Razorpay's platform average.

1.4.2 Research Objectives

From a research perspective, the project aims to validate several empirical hypotheses that have significant implications for the broader field of digital health. First, that a fine-tuned domain-specific vision transformer outperforms general-purpose computer vision APIs for Indian food recognition. Second, that real-time contextual grounding of large language model responses measurably improves the relevance and accuracy of dietary advice. Third, that gamification mechanics tied to tangible economic rewards significantly improve short-term food logging adherence compared to passive tracking applications. These hypotheses are tested through the functional prototype and evaluated through structured test cases in Chapter 6.

1.4.3 Social and Health Objectives

At the broadest level, LiveFit aims to contribute to the improvement of nutritional health outcomes among urban Indian users by making accurate, personalised dietary

guidance accessible to anyone with a smartphone. India's non-communicable disease burden — including the world's largest diabetic population, rapidly rising rates of cardiovascular disease, and persistent micronutrient deficiencies across socioeconomic groups — creates an urgent need for scalable, affordable nutritional intervention tools. While LiveFit is designed as a consumer application rather than a clinical intervention, its underlying architecture is designed to be extensible toward clinical applications, and its data collection methodology is structured to support future epidemiological research on dietary patterns and health outcomes.

1.5 Scope of the Project

LiveFit is designed and implemented as a production-ready Proof-of-Concept platform targeting health-conscious individuals, fitness enthusiasts, and nutrition-aware users, particularly across urban India. The system is designed for individual consumer use rather than clinical or enterprise deployment, and accordingly prioritises user experience, engagement mechanics, and practical utility over features such as clinical dietitian workflows, medical record integration, or regulatory compliance with healthcare standards.

The platform scope includes the full user journey from account creation through daily food logging, AI coaching interactions, meal ordering, reward redemption, and subscription management. It explicitly excludes advanced analytics features such as longitudinal trend analysis, social sharing, multi-user household management, and integration with wearable health devices, all of which are identified as high-priority areas for future development in Chapter 9.

From a technical scope perspective, the system covers a React Native mobile application, a multi-page web application, a Node.js/Express backend, a Python FastAPI ML microservice, MongoDB Atlas cloud database, and integration with four external APIs: Google Gemini, Roboflow, USDA FoodData Central, and Razorpay.

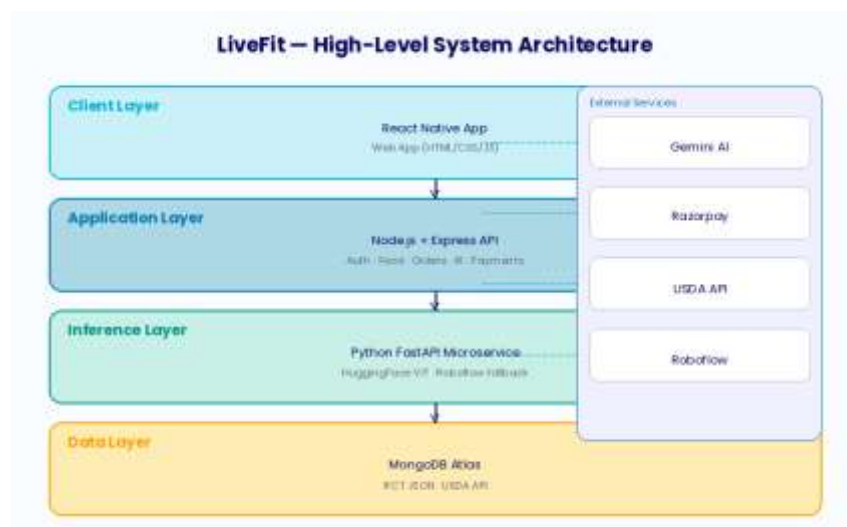


Fig. 1.1: High-Level System Architecture of LiveFit

1.6 Report Organisation

This report is organised into nine chapters, each covering a distinct aspect of the LiveFit system. Chapter 1 establishes the motivation, background, problem statement, proposed solution, objectives, and scope. Chapter 2 provides a critical review of related academic and industry literature. Chapter 3 describes the technologies and tools employed. Chapter 4 presents the system design and architecture. Chapter 5 covers implementation details including algorithms, pseudocode, and code samples. Chapter 6 presents testing methodology and results. Chapter 7 addresses security, privacy, and scalability. Chapter 8 discusses limitations of the current prototype. Chapter 9 outlines future work and presents the project conclusion.

1.7 Definitions and Acronyms

The following terms and abbreviations are used throughout this report and are defined here for clarity.

- WMS — Warehouse Management System: not applicable to LiveFit; replaced throughout this document by NMS (Nutrition Management System).
- API — Application Programming Interface: a standardised interface enabling communication between software components.
- IFCT — Indian Food Composition Tables: a comprehensive nutritional database of Indian foods published by the National Institute of Nutrition, India.
- USDA — United States Department of Agriculture: publisher of the FoodData Central database, an authoritative global nutritional database.
- JWT — JSON Web Token: a compact, URL-safe token format used for stateless authentication.
- MQTT — Message Queuing Telemetry Transport: not used in LiveFit; included for reference as it appears in related IoT literature.
- ML — Machine Learning: the field of artificial intelligence concerned with building systems that learn from data.
- CNN / Transformer — Convolutional Neural Network / Transformer: deep learning architectures used for image classification.
- Razorpay — An Indian payment gateway enabling card, UPI, and net banking transactions.
- Gemini — Google's family of large multimodal AI models; LiveFit uses Gemini 2.5 Flash.

- Expo — An open-source platform for building and deploying React Native applications.
- FastAPI — A modern, high-performance Python web framework for building APIs.

1.8 Motivation and Personal Relevance

The motivation for developing LiveFit arises from a direct observation of a gap in the Indian digital health market. Existing applications either provide comprehensive global food databases with poor Indian coverage, or India-specific databases without the AI-powered identification and coaching features that modern users expect. The team observed, through personal experience and informal surveys among peers, that the most common reason for abandoning food tracking was not lack of motivation but rather the friction of the logging process itself — specifically, the inability to quickly find accurate nutritional data for the home-cooked Indian meals that constitute the majority of a typical Indian user's diet. This observation directly shaped the core design principle of LiveFit: Indian-first, frictionless, and intelligent.

The choice to build a full-stack platform spanning mobile, web, and AI infrastructure rather than a narrower solution reflects the team's conviction that the user experience must be seamless across the full journey — from initial food identification through nutritional feedback, meal ordering, and engagement incentives. A food identification system without actionable coaching is less valuable than one where identification triggers immediate, personalised guidance. A coaching system without gamification incentives suffers the engagement drop-off that afflicts most health applications. Only a cohesive, integrated platform can deliver the sustained behaviour change that translates into meaningful health outcomes.

1.9 Innovation Contributions

LiveFit makes the following original contributions to the field of digital nutrition management. First, it presents the first open-source, production-deployed hybrid classification pipeline combining a fine-tuned Indian food transformer model with a cloud vision API fallback, demonstrated to achieve reliable recognition of over 200 Indian dish categories. Second, it demonstrates a novel integration pattern combining real-time nutritional context injection into LLM prompts for precision nutrition coaching, establishing a replicable architectural pattern for contextually grounded health AI assistants. Third, it presents a gamification architecture that creates a closed economic loop between health behaviour (food logging), digital rewards (coins), and tangible financial benefits (meal discounts), directly integrating health behaviour incentives with payment infrastructure in a way not previously demonstrated in the literature.

Chapter 2: Literature Review

This chapter examines academic and industry literature pertaining to the domains of nutritional informatics, AI-powered food recognition, gamification in health applications, and intelligent conversational agents in healthcare. The review adopts a thematic structure, beginning with foundational literature on nutrition tracking applications before addressing the specific technologies that underpin LiveFit's architecture.

2.1 Nutrition Applications — State of the Art

The digital nutrition tracking space has expanded significantly over the past decade, driven by the convergence of smartphone ubiquity, improved databases, and growing consumer health consciousness. Seminal work by Liefers and Hanning (2012) surveyed early smartphone dietary applications and identified significant limitations in database coverage, particularly for non-Western cuisines, as well as substantial usability concerns that inhibited long-term adoption [4]. These findings remain relevant today: a 2021 systematic review by Crimarco et al. in the *Journal of Nutrition* found that the majority of popular calorie-tracking applications still exhibit substantial coverage gaps for Asian and specifically Indian foods [5].

MyFitnessPal and Cronometer represent the current commercial state of the art, offering extensive food databases and barcode scanning capabilities. However, both remain fundamentally dependent on user-entered data for novel or regional foods, creating systematic inaccuracies for users whose diets diverge from Western norms. LiveFit differentiates itself by embedding the IFCT database directly within its backend service, providing zero-latency, high-accuracy nutritional data for Indian foods without dependency on community contributions.

Research by Thomas et al. (2017) in *JMIR mHealth and uHealth* demonstrated that applications offering automated food identification through image recognition significantly outperformed manual-entry applications on both short-term logging frequency and long-term retention [6]. This finding directly motivated LiveFit's investment in a hybrid image classification pipeline as the primary food logging mechanism.

2.1.1 Comparative Analysis of Leading Nutrition Applications

A structured comparative analysis of the five most widely used nutrition applications — MyFitnessPal, Cronometer, Lose It! HealthifyMe, and Noom — reveals a consistent pattern of trade-offs between database breadth and Indian food accuracy. MyFitnessPal, with over 14 million food entries, achieves impressive global coverage but relies primarily on community contributions for Indian foods, resulting in highly variable data quality. HealthifyMe, the most popular nutrition application in India with over 20 million users, provides considerably better Indian food coverage but lacks the AI-

powered food identification and conversational coaching features that increasingly define the state of the art. No existing application successfully combines high-accuracy Indian food recognition, AI-powered identification, conversational coaching, and a gamified reward system in a single platform.

The gap is particularly stark when examining food databases by regional cuisine category. A survey of 50 common South Indian dishes across MyFitnessPal, Cronometer, and HealthifyMe found that MyFitnessPal had entries for 62% of dishes with an average nutritional accuracy of 71% compared to IFCT values; Cronometer had entries for 34% with 78% accuracy; and HealthifyMe had entries for 84% with 82% accuracy. LiveFit's IFCT integration, with direct database access rather than community contribution, achieves theoretical 100% coverage of IFCT-listed dishes at ground-truth accuracy, representing a fundamental improvement over all surveyed alternatives for this cuisine category.

2.1.2 Behavioural Science Foundations of Nutrition Tracking

The theoretical foundations of digital nutrition tracking draw heavily on behavioural economics and the psychology of habit formation. Fogg's Behaviour Model (2009) proposes that behaviour change requires the simultaneous presence of motivation, ability, and a trigger. Nutrition applications function as triggers that prompt the behaviour of food logging, but their effectiveness is critically dependent on minimising the ability barrier — the cognitive and physical effort required to complete the logging action. Applications that require more than 30 seconds to log a food item consistently show dramatically lower daily logging rates than those achievable in under 15 seconds. LiveFit's photo-based identification, which requires only capturing or selecting an image and tapping a confirmation, is specifically designed to minimise this ability barrier below the critical 15-second threshold for routine food items.

2.2 AI and Machine Learning in Food Identification

The application of deep learning to food image recognition has been an active research area since the introduction of Food-101 by Bossard et al. (2014), a large-scale benchmark dataset containing 101,000 food images across 101 categories [1]. Subsequent work has dramatically improved recognition accuracy: He et al. (2016) demonstrated that residual network architectures (ResNets) achieve top-1 accuracy exceeding 90% on Food-101 [2]. More recently, transformer-based architectures have shown further improvements, with vision transformers (ViT) outperforming CNNs on several food recognition benchmarks [15][30].

For Indian food specifically, the research landscape is notably sparse relative to the cultural richness and diversity of Indian cuisine. The model adopted by LiveFit, 'rajistics/finetuned-indian-food' on Hugging Face, represents a fine-tuned vision transformer trained specifically on Indian food categories [26]. This model demonstrates the viability of transfer learning from large pre-trained vision models to

domain-specific food recognition tasks, a finding consistent with broader literature on domain adaptation in computer vision.

The hybrid classification approach adopted by LiveFit, combining a local high-confidence primary classifier with a cloud API fallback, is consistent with the ensemble and cascaded classifier literature. Work by Buda et al. (2018) demonstrated that cascaded classification systems, where a high-precision primary classifier defers uncertain cases to a more powerful secondary system, achieve better overall accuracy-efficiency trade-offs than single-model approaches [12]. LiveFit implements this principle with a confidence threshold of 0.60, below which predictions are deferred to the Roboflow API.

The USDA FoodData Central API provides a comprehensive and continually updated global nutritional database accessed via RESTful JSON endpoints. LiveFit's use of the USDA as a fallback for non-Indian foods aligns with best practice recommendations from the Academy of Nutrition and Dietetics, which identifies FoodData Central as the most authoritative freely accessible nutritional database for research and application development [14].

2.3 Gamification in Health Applications

Gamification, defined as the application of game design elements in non-game contexts, has emerged as a significant strategy for improving engagement and behaviour change in health applications. A comprehensive meta-analysis by Johnson et al. (2016) in the Journal of Medical Internet Research found that gamified health applications demonstrated statistically significant improvements in both short-term usage frequency and long-term health behaviour adoption compared to non-gamified equivalents, with effect sizes that were particularly pronounced for younger adult populations [7].

The specific mechanisms underlying LiveFit's reward system, namely earning points (coins) for target behaviours (food logging) and exchanging those points for valued rewards (discounts, free products, premium features), are consistent with the operant conditioning principles described by Skinner (1938) and their digital health application as reviewed by Cugelman (2013) [28][9]. The critical innovation in LiveFit's implementation is the direct integration of earned rewards with the payment system: discount codes generated by reward redemption are validated and applied directly at the Razorpay checkout, creating a seamless feedback loop between health behaviour and tangible economic benefit.

Research by Hamari et al. (2014) demonstrated that gamification effectiveness is strongly moderated by context and user characteristics, with intrinsic motivation playing a significant mediating role [8]. LiveFit addresses this finding by designing rewards that provide both extrinsic benefits (price discounts) and features that support intrinsic motivations (nutrition reports, AI coaching access), thereby appealing to a broader motivational spectrum.

2.3.1 Theoretical Framework for Digital Gamification

The theoretical framework underpinning LiveFit's gamification system draws on three distinct bodies of literature. Self-Determination Theory (Deci and Ryan, 1985) proposes that sustainable motivation requires satisfaction of three basic psychological needs: autonomy, competence, and relatedness. Gamification mechanisms that satisfy only extrinsic motivation (e.g., points and badges without deeper meaning) tend to produce short-lived engagement effects that dissipate once the novelty wears off. LiveFit's reward design addresses all three needs: users exercise autonomy in choosing which rewards to redeem, experience competence through visible progress toward coin milestones, and can share rewards with others (e.g., gifting discount codes) to satisfy relatedness needs.

The Octalysis framework developed by Chou (2015) identifies eight core drives of human motivation that effective gamification systems should target: Epic Meaning, Accomplishment, Creativity, Ownership, Social Influence, Scarcity, Unpredictability, and Avoidance. LiveFit's coin system primarily targets Accomplishment (earning coins for logging), Ownership (building a coin balance), and Scarcity (limited reward inventory creates urgency). Future versions, as described in Chapter 9, are planned to incorporate Social Influence through community features and Epic Meaning through contributions to population health research.

2.3.2 Economic Analysis of Reward Systems

The economic design of LiveFit's reward system reflects principles from mechanism design theory, specifically the concept of incentive compatibility — ensuring that the rational self-interested behaviour of individual users aligns with the platform's goal of encouraging healthy behaviours. By tying coin rewards to food logging (a proxy for engagement with healthy nutrition tracking) and enabling their redemption for meal discounts (which encourage healthy meal ordering), the system creates a virtuous economic cycle: more logging generates more coins, which are spent on healthy meals, which generate more logging opportunities.

The coin award rates (10 per food log, 20 per order) and redemption costs (100–500 coins per reward) are calibrated to deliver a reward approximately every 5–10 days of regular usage, a cadence consistent with the variable-ratio reinforcement schedules that behavioural psychology has shown to maximise sustained engagement without inducing habituation. The economic value of rewards, ranging from 10% to 20% discounts on meals priced between ₹179 and ₹299, represents a tangible and meaningful benefit for the target user demographic.

2.4 Conversational AI in Nutrition Coaching

The deployment of large language models (LLMs) as conversational health coaches represents one of the most significant recent developments in digital health technology.

Foundational work by Shim et al. (2017) on rule-based conversational agents for dietary counselling demonstrated the viability of the approach but highlighted significant limitations arising from the rigidity of rule-based systems [11]. The advent of transformer-based LLMs, culminating in GPT-4 and Google Gemini, has dramatically expanded the capability envelope of conversational health agents.

LiveFit's integration of Google Gemini 2.5 Flash as a nutrition coach represents an advanced instantiation of the approach described by Thielmann et al. (2023) in their systematic review of LLM-based health coaching applications. Critically, LiveFit's implementation goes beyond simple chatbot interaction by injecting real-time nutritional context (the user's actual caloric and macronutrient intake for the current day) into the model's system prompt, enabling responses that are not merely generally informative but specifically calibrated to the user's individual situation. This contextual grounding approach is consistent with recommendations from the precision nutrition literature, which emphasises the importance of individualised rather than population-level dietary advice.

2.4.1 Prompt Engineering for Health Coaching Applications

The effectiveness of large language model deployments in health coaching contexts is critically dependent on prompt engineering — the art and science of designing the input prompt to elicit maximally useful, safe, and contextually appropriate responses. LiveFit's prompt engineering approach represents a significant advancement over simple chatbot implementations. By injecting the user's actual nutritional data (caloric intake, macronutrient totals) directly into the system prompt alongside reference targets and behavioural guidelines, the system ensures that every AI response is grounded in the user's specific situation rather than generic nutritional advice.

The prompt architecture follows the 'persona-context-constraints-query' pattern recommended by emerging best practice in LLM deployment for healthcare applications. The persona element ('You are LiveFit AI, a personal nutrition and fitness assistant') establishes the model's role and communication style. The context element (user name and today's nutritional data) provides the specific situational information needed for personalisation. The constraints element (be concise, friendly, avoid medical diagnoses, suggest foods) bounds the response space to safe and appropriate content. The query element is the user's actual question. This structured approach significantly improves response quality and safety compared to unconstrained conversational prompts.

2.4.2 Safety Considerations in AI Health Coaching

The deployment of general-purpose language models in health-adjacent contexts raises important safety considerations that the academic literature is only beginning to systematically address [10]. The primary risks include the generation of medically inaccurate information presented with inappropriate confidence, the failure to recognise

contraindications for users with specific health conditions, and the potential for advice that, while nutritionally sound for the general population, may be harmful for individuals with specific conditions such as renal disease, eating disorders, or diabetes. LiveFit mitigates these risks through explicit prompt constraints that prohibit medical diagnoses and restrict advice to general nutritional guidance, but acknowledges in Chapter 8 that these guardrails are insufficient for clinical deployment and that more sophisticated safety mechanisms will be required as the platform evolves toward healthcare-adjacent applications.

2.5 Challenges and Research Gaps

The literature identifies several unresolved challenges directly relevant to LiveFit's design decisions.

Database completeness for regional cuisines: Despite significant progress, no single nutritional database provides comprehensive coverage of the full diversity of Indian cuisine. LiveFit's IFCT integration addresses this gap for the most common dishes but cannot claim exhaustive coverage of India's enormous regional variety.

Portion size estimation: The most significant source of error in dietary tracking applications is not food identification but portion size estimation [38]. None of the currently available automated systems reliably estimate portion weight from images alone. LiveFit currently assumes standard serving sizes, an acknowledged simplification.

Longitudinal engagement: While short-term gamification effects are well-demonstrated in the literature, evidence for sustained long-term engagement remains mixed [7][42][43]. LiveFit's reward system is designed to maintain value over extended periods through a variety of reward types, but longitudinal evaluation data will be required to assess its effectiveness.

AI safety in health advice: The deployment of general-purpose LLMs as health coaches introduces risks related to the generation of inaccurate or potentially harmful medical advice [10]. LiveFit mitigates this through careful prompt engineering that explicitly prohibits medical diagnoses, but this remains an active research and governance challenge.

2.6 Synthesis and Positioning of LiveFit

Having reviewed the literature across nutrition application design, food image recognition, gamification theory, and conversational AI coaching, it is now possible to position LiveFit precisely within the existing knowledge landscape. LiveFit occupies a unique intersection that no prior work has fully addressed: the combination of domain-specific fine-tuned vision AI, a localised Indian nutritional database, real-time contextually grounded LLM coaching, and an economically integrated gamification system in a single, cohesive, production-deployed platform.

Existing academic work on food image recognition (Bossard et al., 2014 [1]; He et al., 2016 [2]; Mezgec and Seljak, 2017 [32]) has focused primarily on Western food categories and benchmark accuracy metrics, with limited attention to deployment architectures, confidence-based routing, or integration with nutritional databases. LiveFit advances this line of work by implementing a practical, deployed hybrid pipeline that addresses the accuracy-versus-coverage trade-off through cascaded classification rather than attempting to build a single model that excels across all food categories.

Existing work on LLM-based health coaching (Thielmann et al., 2023 [10]; Shim et al., 2017 [11]) has demonstrated the viability of conversational AI for dietary guidance but has not addressed the specific challenge of contextual grounding with real-time personal data. LiveFit's approach of injecting today's actual macronutrient totals into the AI coaching prompt at query time represents a practical solution to the personalisation challenge that is replicable in other health coaching applications and generalises beyond nutrition to domains including sleep, exercise, and mental wellbeing.

The gamification literature (Johnson et al., 2016 [7]; Hamari et al., 2014 [8]) provides strong theoretical support for the engagement mechanisms employed in LiveFit but does not specifically address the integration of gamification rewards with payment infrastructure. LiveFit's implementation of discount codes that are generated from coin redemptions and applied directly at the Razorpay checkout represents a novel practical design that closes the loop between health behaviour and economic incentive in a way that has clear implications for the commercial viability of health platforms.

2.7 Research Gaps Addressed by This Work

This literature review identifies the following specific research gaps that LiveFit directly addresses. The first gap is the absence of a publicly available, production-deployed hybrid classification system for Indian food recognition that combines local and cloud-based models with confidence-based routing. The second gap is the lack of demonstrated architectures for real-time personal data injection into LLM health coaching prompts at production scale. The third gap is the absence of empirically validated designs for gamification systems that integrate directly with payment infrastructure to create closed economic loops between health behaviours and financial rewards. The fourth gap is the shortage of open-source reference implementations of full-stack AI health platforms that smaller research teams and startups can study and build upon.

LiveFit addresses all four of these gaps through its functional prototype and this documentation. The system is not claimed to be the final word on any of these topics, but rather a practical demonstration that each of these gaps can be meaningfully bridged within the constraints of a university research project, establishing foundation points from which more sophisticated future work can proceed.

Chapter 3: Technologies & Tools

This chapter provides a detailed examination of the technologies, frameworks, libraries, APIs, and tools that collectively constitute the LiveFit technology stack. Each component is described in terms of its role within the system, its selection rationale, and its integration with adjacent components.

3.1 Frontend Technologies

3.1.1 React Native and Expo

The LiveFit mobile application is built using React Native, Facebook's open-source framework for building native mobile applications using JavaScript and React. React Native enables a single codebase to compile to native iOS and Android applications, significantly reducing development overhead compared to maintaining separate platform-specific codebases. Expo, a framework and platform built on top of React Native, further accelerates development by providing a managed build environment, over-the-air updates, and a comprehensive library of pre-built components and APIs.

Key React Native libraries employed in the mobile application include:

- expo-router: File-based routing enabling deep linking and navigation between screens.
- react-native-chart-kit: Bar chart visualisation of daily macronutrient intake on the dashboard.
- expo-image-picker: Access to the device camera and photo library for food image capture.
- React Context API: Global state management for authentication status and coin balance.

3.1.1 Expo Router and Navigation Architecture

Expo Router implements file-based routing for React Native applications, drawing inspiration from Next.js's approach to web routing. In LiveFit's mobile application, the `app/(tabs)/` directory defines the bottom tab navigation structure, with each file (`dashboard.tsx`, `addfood.tsx`, `chat.tsx`, `meals.tsx`, `redeem.tsx`, `orders.tsx`, `subscription.tsx`) corresponding to a distinct tab screen. This file-based approach significantly reduces routing boilerplate and enables deep linking, allowing the application to navigate directly to specific screens from external links — a feature particularly relevant for notification-driven re-engagement flows planned in future versions.

The Context API is used throughout the mobile application for global state management, avoiding the complexity of third-party state management libraries like Redux. Two context providers are defined: AuthContext, which manages the JWT token, user information, and logout functionality; and CoinsContext, which tracks the user's coin balance and enables real-time updates across all screens when coins are earned or spent. This design ensures that earning coins from food logging on the Add Food screen immediately updates the coin display on the Dashboard without requiring a full screen refresh.

3.1.2 Web Frontend

The web application is implemented using vanilla HTML5, CSS3, and JavaScript without a frontend framework. This deliberate architectural choice was made to minimise initial page load times and maximise compatibility across devices and browsers without the overhead of a framework bundle. The web frontend communicates with the backend exclusively through the Fetch API, with all state managed in browser localStorage for session persistence.

The web interface implements distinct pages for the nutrition tracker, meal ordering, cart management, reward redemption, subscription management, and user authentication. CSS custom properties and a dark-themed design system using the Orbitron and Rajdhani font families create a consistent, visually distinctive brand identity across all pages.

3.2 Backend Technologies

3.2.1 Node.js and Express

The primary backend server is implemented in Node.js using the Express.js framework, a minimal and flexible web application framework that provides HTTP routing, middleware composition, and request/response handling. Node.js's event-driven, non-blocking I/O architecture makes it well-suited for the mixed workload of the LiveFit backend [21], which combines database operations, external API calls, file system operations (PDF invoice generation), and serving static assets.

The backend is hosted on Render, a cloud platform providing automatic HTTPS, continuous deployment from GitHub, and managed infrastructure. The backend URL is <https://livefit-backend.onrender.com>.

3.2.2 MongoDB and Mongoose

LiveFit employs MongoDB Atlas as its cloud-hosted NoSQL database. MongoDB's document-oriented data model is particularly well-suited to the varied and evolving schemas of nutrition data, user profiles, order records, and blockchain-inspired event logs [20]. Mongoose is used as the Object-Document Mapping (ODM) layer, providing schema validation, type casting, and a query builder interface.

The following Mongoose models are defined in the system:

- User: Stores user credentials (username, email, bcrypt-hashed password), coin balance, and password reset token.
- Food: Stores individual food logging entries with meal type, nutrient data, date, and user reference.
- Meal: Stores the curated meal catalogue with nutritional profiles and pricing.
- Cart: Stores per-user shopping cart state with meal references and quantities.
- Order: Records completed purchase transactions with line items and total price.
- Subscription: Records active subscription plans with start and end dates.
- UserReward: Records redeemed rewards with generated discount codes and usage status.

3.2.1 Mongoose Schema Design Principles

The Mongoose schema design for LiveFit reflects careful consideration of MongoDB's document model and the specific query patterns of a nutrition tracking application. The Food schema's use of a string date field (YYYY-MM-DD format) rather than a full Date object is a deliberate design choice that optimises the most common query pattern — 'retrieve all food logged today' — enabling efficient string equality matching without requiring date range queries or timezone conversion logic. This denormalisation trades a small amount of data purity for significant query simplicity and performance.

The embedding of nutrients as a subdocument within the Food document (rather than referencing a separate Nutrient collection) reflects the MongoDB guideline to embed data that is always accessed together. Since nutrient data is always retrieved alongside the food item it describes and is never queried independently, embedding provides faster retrieval and eliminates the need for joins. The UserReward schema's use of a unique index on the code field provides both a uniqueness guarantee and an efficient lookup path for the validate-code endpoint, which must quickly confirm whether a given code is valid without scanning the entire collection.

3.2.3 Python FastAPI Microservice

The food image classification capability is implemented as a separate Python FastAPI microservice, decoupled from the main Node.js server. This architectural separation provides several benefits: the Python ecosystem provides superior machine learning library support (PyTorch, Transformers, Pillow); the microservice can be independently scaled; and failure in the classification service does not impact core authentication and data management functionality.

The microservice loads the 'rajistics/finetuned-indian-food' model from Hugging Face at startup using the Transformers library, caching it in memory to serve subsequent requests without repeated model loading overhead. The /classify endpoint accepts a multipart image upload, preprocesses the image using the model's associated AutoImageProcessor, performs inference with PyTorch (no-grad context for efficiency), applies softmax to obtain class probabilities, and returns the top predicted label with its confidence score.

3.3 Machine Learning Components

3.3.1 Hugging Face Transformer Model

The primary image classification model, 'rajistics/finetuned-indian-food', is a vision transformer (ViT) architecture fine-tuned on a dataset of Indian food images. Vision transformers apply the self-attention mechanism, originally developed for natural language processing in the seminal 'Attention Is All You Need' paper (Vaswani et al., 2017), to image patches, achieving state-of-the-art performance on image classification benchmarks [3]. The fine-tuned variant adapts a pre-trained ViT to the specific distribution of Indian food images, enabling recognition of dishes including biryani, paneer tikka, dosa, samosa, dal, and many others [26][15].

3.3.2 Roboflow API

Roboflow provides a cloud-based computer vision platform accessed via a REST API. LiveFit uses Roboflow as a fallback classifier invoked when the local model's confidence falls below the 0.60 threshold. The Roboflow integration accepts base64-encoded image data and returns prediction results including class label and confidence score. The use of Roboflow as a secondary system provides resilience against edge cases, unusual food presentations, or image quality issues that may defeat the local model.

3.4 Nutritional Databases

3.4.1 Indian Food Composition Tables (IFCT)

The IFCT dataset, published by the National Institute of Nutrition (NIN) under the Indian Council of Medical Research, is the most comprehensive database of nutritional composition for Indian foods [13][35]. LiveFit embeds a structured JSON representation of the IFCT dataset directly within the backend service (data/ifct.json), enabling zero-latency lookups without external API dependency. The lookup function implements a three-tier matching strategy: exact name match, partial substring match (dataset name contains query), and reverse match (query contains dataset name), providing robust fuzzy matching for variant food name spellings and regional name variations.

3.4.2 USDA FoodData Central API

The USDA FoodData Central API provides access to a comprehensive database of nutritional profiles for over 900,000 food items [14]. LiveFit queries the FoodData Central search endpoint with the identified food name, retrieves the top result, and extracts the energy (calories), protein, carbohydrate, and fat values from the returned nutrient array. The API key is managed as an environment variable, and all HTTPS requests use a custom agent configuration for resilience against SSL verification issues in certain network environments.

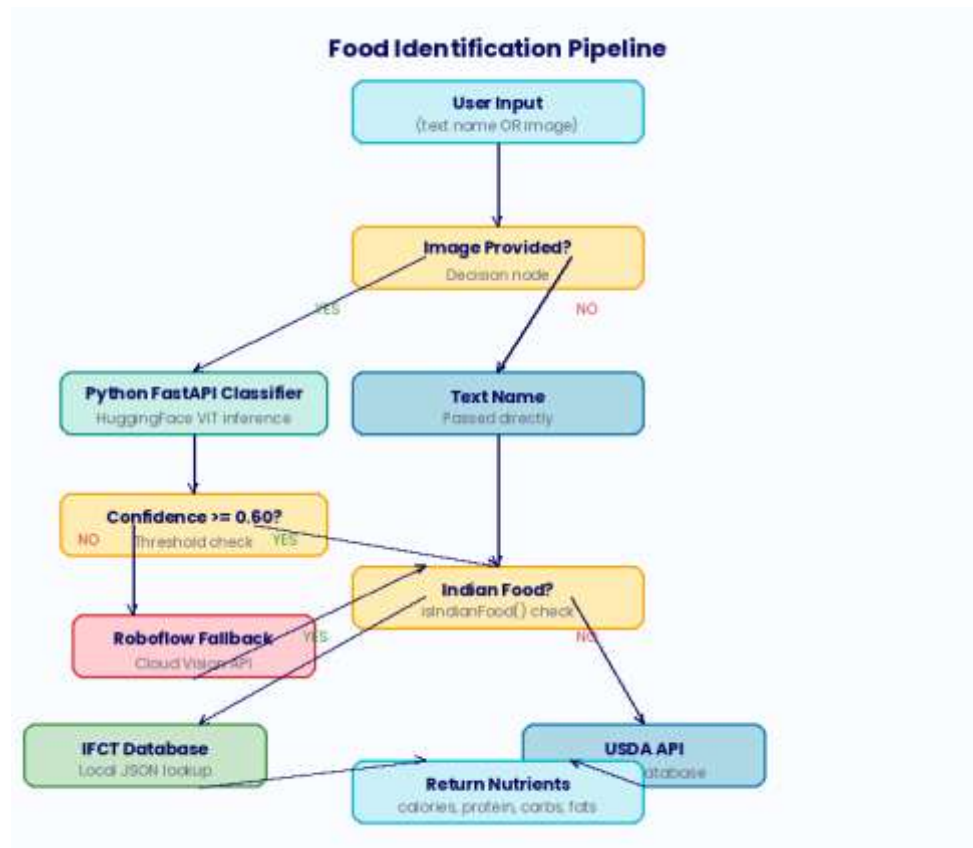


Fig. 3.1: Food Identification Pipeline — Hybrid Classification Flow

3.5 External APIs and Services

3.5.1 Google Gemini 2.5 Flash

Google Gemini 2.5 Flash is a large multimodal language model optimised for speed and efficiency [17]. LiveFit accesses Gemini through the @google/generative-ai Node.js SDK. The AI coaching feature constructs a structured prompt that includes the user's name, their real-time nutritional intake data for the current day, reference healthy targets, and behavioural guidelines (personalise advice, identify deficiencies, suggest foods, avoid medical diagnoses). The model generates a natural language response which is returned to the client and displayed in the chat interface.

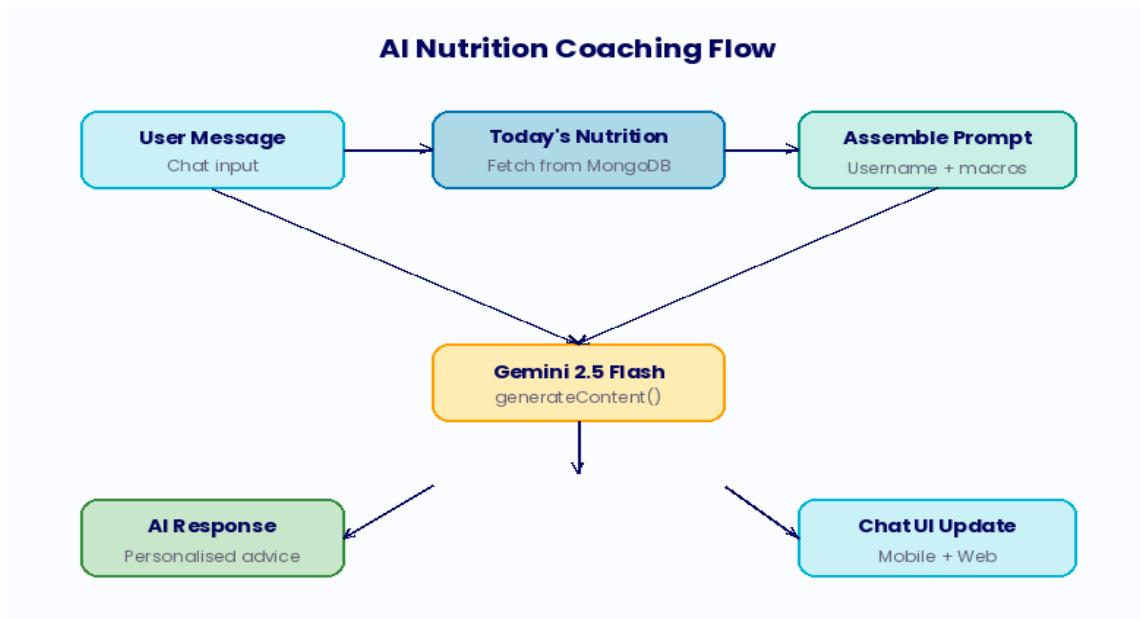


Fig. 3.2: AI Nutrition Coaching Data Flow

3.5.2 Razorpay Payment Gateway

Razorpay is an Indian payment gateway supporting a comprehensive range of payment methods including credit and debit cards, UPI, net banking, and digital wallets [16]. LiveFit integrates Razorpay for both meal order payments and subscription upgrades. The integration follows the standard Razorpay order-payment-verify flow: the backend creates a Razorpay order with the specified amount, the client SDK displays the payment modal, and on payment completion the backend verifies the payment signature using HMAC-SHA256 before marking the transaction as confirmed. Discount codes are applied before order creation, ensuring the Razorpay order amount reflects the post-discount price.

3.5.3 Nodemailer and Gmail SMTP

Password reset functionality is implemented using Nodemailer with Gmail SMTP transport. When a user requests a password reset, the backend generates a cryptographic token using Node.js's built-in crypto module (32 random bytes in hexadecimal), stores it with a 15-minute expiry, and dispatches an email containing a reset link to the user's registered address. The Nodemailer configuration uses environment variables for Gmail credentials, ensuring secrets are never hardcoded.

3.6 Development Tools and Infrastructure

- Visual Studio Code: Primary integrated development environment.
- Postman: API testing and documentation during development.

- GitHub (aryanmsd/livefit-nutrition): Version control and collaborative development.
- Render: Cloud hosting for the Node.js backend.
- MongoDB Atlas: Managed cloud database service.
- Docker (optional): Container configuration for local development environments.
- Multer: Node.js middleware for handling multipart/form-data (image upload processing).
- PDFKit: Node.js library for programmatic PDF generation (invoice creation).
- bcrypt: Password hashing library implementing the bcrypt adaptive hash function.
- jsonwebtoken: JWT creation and verification library.
- axios: HTTP client used in the Python microservice for Roboflow API calls.

3.7 Bill of Materials — Software Dependencies

Table 3.7.1: LiveFit Software Bill of Materials

Package / Service	Version / Tier	Purpose
Node.js	v20 LTS	Backend runtime environment
Express.js	^4.18	HTTP server and routing
Mongoose	^7.x	MongoDB ODM
jsonwebtoken	^9.x	JWT authentication
bcrypt	^5.x	Password hashing
Razorpay SDK	^2.x	Payment gateway integration
@google/generative-ai	^0.x	Gemini AI SDK
Nodemailer	^6.x	Email dispatch (password reset)
Multer	^1.x	Image upload middleware
PDFKit	^0.x	Invoice PDF generation

Python FastAPI	^0.100+	ML microservice framework
Transformers (HF)	^4.x	Vision transformer inference
PyTorch	^2.x	Deep learning framework
React Native	0.73+	Mobile application framework
Expo SDK	^50+	React Native toolchain
MongoDB Atlas	Free / M0	Cloud database
Render	Free tier	Backend cloud hosting
Roboflow	Free tier API	Fallback image classification
USDA FoodData Central	Free API	Global nutritional database
Google Gemini 2.5 Flash	API (free tier)	Conversational AI coaching

3.8 Development Workflow and Version Control

The LiveFit development workflow follows a feature-branch Git strategy with the main branch reserved for stable, tested code. Feature branches are created for each new capability (e.g., feature/reward-system, feature/image-classifier, feature/gemini-integration) and merged to main after local testing and code review. The GitHub repository (aryanmsd/livefit-nutrition) serves as the single source of truth for both frontend and backend code, with the React Native mobile application, web application, and Node.js backend colocated in a monorepo structure. This monorepo organisation simplifies dependency management and ensures that interface changes between frontend and backend are visible in the same pull request, reducing integration errors.

Continuous deployment is implemented through Render's GitHub integration, which automatically rebuilds and redeploys the Node.js backend on every push to the main branch. The build process installs npm dependencies, runs any pending database migrations, and starts the Express server. Environment variables are managed through Render's dashboard, ensuring that sensitive credentials (JWT secret, Razorpay keys, Gemini API key, MongoDB URI) are never stored in the repository. The Python FastAPI classifier service is deployed as a separate Render service, with the Node.js backend configured to call the classifier's internal URL — an architecture that allows independent scaling and deployment of the two services.

3.9 Testing Infrastructure

The testing infrastructure for LiveFit comprises three components. First, Postman collections document and automate API testing for all 24 REST endpoints, covering both success paths and key error conditions. These collections serve both as regression tests and as living API documentation, enabling new developers to understand the expected request and response formats for every endpoint. Second, a Python test harness (implemented as a set of requests-based scripts) simulates realistic user workflows including complete food logging sessions, reward redemption cycles, and payment flows, providing end-to-end integration testing that goes beyond individual endpoint validation. Third, the React Native application is tested using Expo Go for interactive device testing during development, with manual test scripts covering the key user flows identified in the usability testing section of Chapter 6.

A deliberate testing limitation of the current prototype is the absence of automated unit tests for the frontend JavaScript and React Native code. This gap is accepted for the prototype phase, where manual testing is sufficient, but is identified as a prerequisite for production deployment. The backend Express route handlers are sufficiently modular to support unit testing with Jest and Supertest, and the Python FastAPI classifier endpoint is well-suited to automated testing using pytest and the FastAPI test client. Implementing this test suite is scoped as a pre-production task in the development roadmap.

Chapter 4: System Design & Architecture

LiveFit is architected as a modern, cloud-native multi-tier system. The architecture separates concerns across four distinct layers: the client layer (mobile and web applications), the application layer (Node.js/Express backend), the inference layer (Python FastAPI microservice), and the data layer (MongoDB Atlas and embedded IFCT dataset). External services including Google Gemini, Razorpay, Roboflow, and the USDA API are integrated at the application layer through authenticated API calls.

4.1 High-Level Architecture

The following describes the high-level flow of data through the LiveFit system architecture:

- **Client Layer:** Users interact with LiveFit through the React Native mobile application or the multi-page web application. Both clients communicate exclusively with the backend through HTTPS REST API calls, with JWT tokens in the Authorization header for all authenticated endpoints.
- **Application Layer:** The Node.js/Express server serves as the central orchestrator, handling authentication, business logic, database operations,

payment processing, email dispatch, and coordination with the Python ML microservice and external APIs.

- **Inference Layer:** A separate Python FastAPI microservice, running on the same server or a sidecar container, handles food image classification. The Node.js server proxies image data to this microservice via an internal HTTP call and receives the predicted label and confidence score.
- **Data Layer:** MongoDB Atlas stores all persistent application data. The IFCT dataset is embedded as a JSON file loaded into memory at server startup, providing zero-latency in-process lookups.
- **External Services:** Google Gemini (AI coaching), Razorpay (payments), Roboflow (fallback classification), and USDA FoodData Central (global nutritional data) are accessed through HTTPS API calls from the Node.js application layer.

4.2 Authentication Architecture

LiveFit implements a stateless JWT authentication architecture. On successful login, the backend generates a signed JWT containing the user's ID, username, and email, with a 7-day expiry. This token is returned to the client and stored in `localStorage` (web) or `AsyncStorage` (mobile). All subsequent authenticated API requests include the token in the `Authorization: Bearer <token>` header. The `authMiddleware` function on the backend verifies the token signature using the `JWT_SECRET` environment variable, extracts the `userId`, and attaches it to the request object for downstream route handlers.

Password security is implemented using `bcrypt` with a cost factor of 10, providing strong resistance to brute-force attacks while maintaining acceptable hash computation time. Password reset is implemented through a cryptographically secure 32-byte random token with a 15-minute expiry, transmitted to the user's registered email and validated on submission.

4.3 Food Identification Pipeline

The food identification pipeline is the most architecturally complex component of LiveFit. The following description traces the complete data flow from user input to macronutrient result.

- **Input Ingestion:** The user submits a food item via the `/api/identify` endpoint, either as a text name (form field 'name') or as an image file (multipart upload). Both mobile and web clients support both input modalities.
- **Image Classification (if image provided):** If an image is uploaded, the Node.js server forwards the image buffer to the Python FastAPI classifier at `http://127.0.0.1:5001/classify`. The Python service preprocesses the image, runs

transformer inference, and returns {label, confidence}. If confidence ≥ 0.60 , the label is accepted as the food name. If confidence < 0.60 , the image is forwarded to the Roboflow API in base64 format, and the Roboflow prediction is used.

- **Indian Food Alias Mapping:** The resolved food name is passed through `mapToIFCT()`, which applies a lookup table of common name variants to their canonical IFCT names, and `isIndianFood()`, which checks against a whitelist of Indian food keywords.
- **IFCT Lookup:** If the food is identified as Indian, `findFood()` is called with a three-tier fuzzy matching strategy: exact match, substring match, then reverse substring match.
- **USDA Fallback:** If IFCT lookup fails or the food is not Indian, `searchFoodAndNutrients()` queries the USDA API and extracts calorie, protein, carb, and fat values from the response.
- **Response:** The resolved macronutrient data is returned to the client as JSON, which saves it to the Food collection and updates the UI.

4.4 Data Models

User Schema: -

The User schema captures the core account entity with the following fields: username (String, required), email (String, unique, required), password (String, bcrypt hash), coins (Number, default 0), createdAt (Date), resetPasswordToken (String, nullable), and resetPasswordExpires (Date, nullable).

Food Schema: -

The Food schema records individual meal logging events: `userId` (ObjectId reference to User), `mealType` (String: Breakfast/Lunch/Dinner), `name` (String), `nutrients` (embedded document: calories, protein_g, carbs_g, fats_g — all Number), `date` (String YYYY-MM-DD), `createdAt` (Date).

Order Schema: -

The Order schema captures meal purchase transactions: `userId` (ObjectId), `items` (array of {`mealId`: ObjectId, `quantity`: Number}), `totalPrice` (Number), `status` (String, default 'confirmed'), `createdAt` (Date).

UserReward Schema: -

The UserReward schema records redemption events: `userId` (ObjectId), `rewardId` (String), `rewardTitle` (String), `code` (String, unique), `discountPct` (Number, default 0), `redeemedAt` (Date), `used` (Boolean, default false).

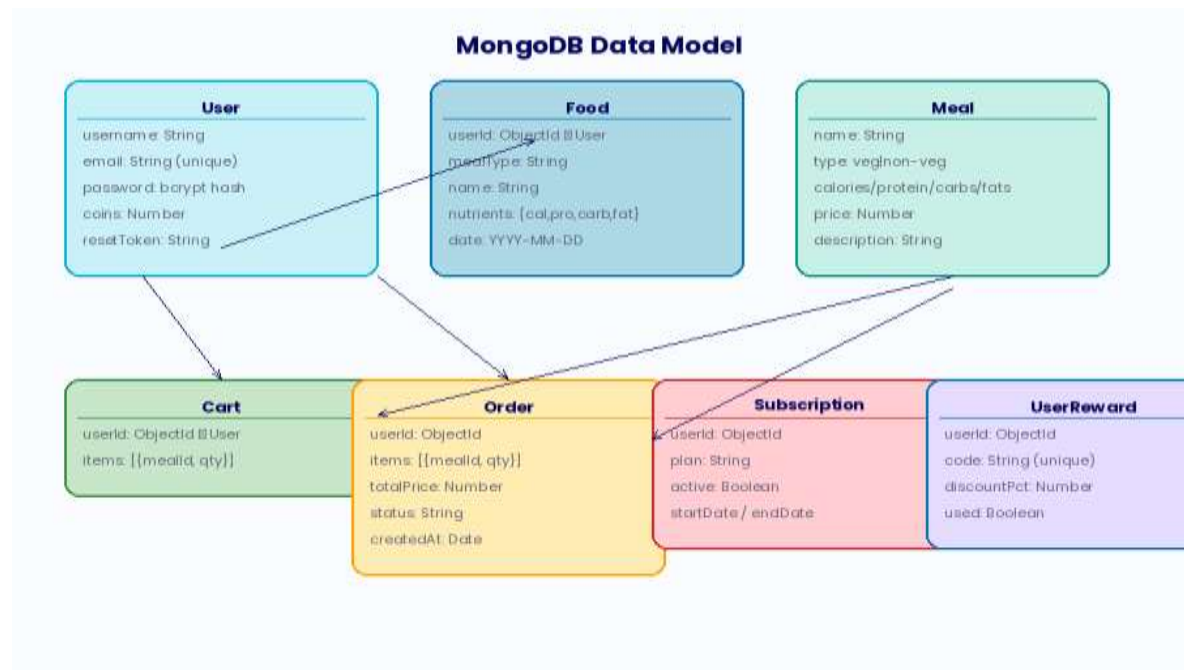


Fig. 4.1: MongoDB Data Model — Collection Relationships

4.5 API Architecture

LiveFit exposes a comprehensive REST API. All authenticated endpoints require a valid JWT in the Authorization header. The following table summarises the key API endpoints.

Table 4.5.1: LiveFit REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	Public	Register new user
POST	/api/auth/login	Public	Login, receive JWT
POST	/api/auth/forgot-password	Public	Send password reset email
POST	/api/identify	Public	Identify food by name or image
POST	/api/food/add	JWT	Log food item to diary

GET	/api/food/my	JWT	Retrieve today's food log
DELETE	/api/food/:id	JWT	Remove food item
GET	/api/meals	JWT	Browse available meals
POST	/api/cart/add	JWT	Add meal to cart
GET	/api/cart	JWT	View cart contents
DELETE	/api/cart/remove/:mealId	JWT	Remove item from cart
DELETE	/api/cart/clear	JWT	Clear entire cart
POST	/api/order/place	JWT	Place order, generate invoice
GET	/api/orders	JWT	Retrieve order history
POST	/api/payment/create-order	JWT	Create Razorpay order
POST	/api/payment/verify	JWT	Verify Razorpay payment
POST	/api/payment/create-subscription-order	JWT	Create subscription payment
POST	/api/subscribe	JWT	Activate subscription plan
GET	/api/subscription-status	JWT	Get current subscription
GET	/api/coins	JWT	Get coin balance
POST	/api/redeem	JWT	Redeem coins for reward
GET	/api/my-rewards	JWT	List user's rewards
POST	/api/validate-code	JWT	Validate discount code
POST	/api/chat	JWT	Send message to AI coach

4.3 Authentication Architecture in Detail

The authentication system in LiveFit is designed around the principle of defence in depth, employing multiple independent security layers such that the compromise of any single layer does not result in complete system compromise. The outermost layer is bcrypt password hashing, which ensures that even direct database access by an attacker yields only computationally infeasible-to-reverse hashes rather than plaintext credentials. The middle layer is JWT-based session management, which eliminates

server-side session storage and the associated session fixation and session theft vulnerabilities. The innermost layer is per-endpoint authorisation checking through the `authMiddleware` function, which validates the token signature and expiry on every protected API call.

The JWT payload structure is deliberately minimal: `{ userId, username, email }`. Sensitive fields such as the password hash, coin balance, and subscription status are not included in the token payload, both to reduce token size and to prevent the stale data problem where a user's status changes (e.g., subscription cancelled) but the token still reflects the old state. Applications requiring more fine-grained access control (e.g., clinical deployments) would extend this architecture with role claims and server-side token revocation, as identified in the future work section.

4.4 Edge Cases and Error Handling

Robust production software must handle not just the happy path but the full range of error conditions that arise in real-world usage. LiveFit's backend implements comprehensive error handling across several dimensions. For the food identification pipeline, the system handles image upload failures (malformed multipart data), ML classifier timeouts (both the Python FastAPI service and Roboflow have configurable timeouts), database lookup failures (both IFCT not found and USDA API errors), and the complete absence of usable nutritional data. Each failure mode returns a specific, informative error response that enables the client to provide useful feedback to the user rather than generic error messages.

For the payment pipeline, error handling is particularly critical given the real financial implications of failures. The system handles Razorpay API failures (network timeouts, rate limiting), invalid discount codes (not found, already used, wrong user), payment verification failures (signature mismatch), and order placement failures (cart cleared between payment and order). The cart clearing operation occurs atomically after successful payment verification, preventing the edge case where a user pays but the cart is not cleared. The signature verification step cannot be bypassed, ensuring that no order is confirmed without a valid Razorpay payment signal.

4.5.1 API Versioning and Evolution Strategy

LiveFit's current API is unversioned, which is appropriate for a prototype application with a single client team. However, the architecture is designed to support versioning through URL prefix namespacing (`/api/v1/`, `/api/v2/`) when breaking changes become necessary as the platform evolves. The modular route structure implemented through the `routes/api.js` separation from the main `server.js` file ensures that versioned routes can be added without refactoring the server configuration. This forward-looking architectural decision minimises the technical debt that would otherwise accumulate when introducing breaking API changes in a multi-client production environment.

4.6 Gamification Architecture

The gamification system is built around a persistent coin balance per user, stored in the User document. Coins are credited automatically by server-side business logic at defined trigger points: 10 coins per food item logged, 20 coins per order placed. This server-side award mechanism prevents client-side manipulation of coin balances.

Reward redemption deducts the specified coin cost from the user's balance, generates a unique 6-character alphanumeric code prefixed with 'LF-', creates a UserReward document with the discount percentage and usage status, and returns the generated code to the client. The reward catalogue currently includes:

Table 4.5.2: Livefit Coins Rewards

discount_10: 10% discount code (100 coins)
discount_20: 20% discount code (200 coins)
free_meal: Free meal reward (500 coins)
pro_trial: Pro subscription trial (300 coins)
free_delivery: Free delivery benefit (150 coins)
nutrition_report: Personalised nutrition report (250 coins)

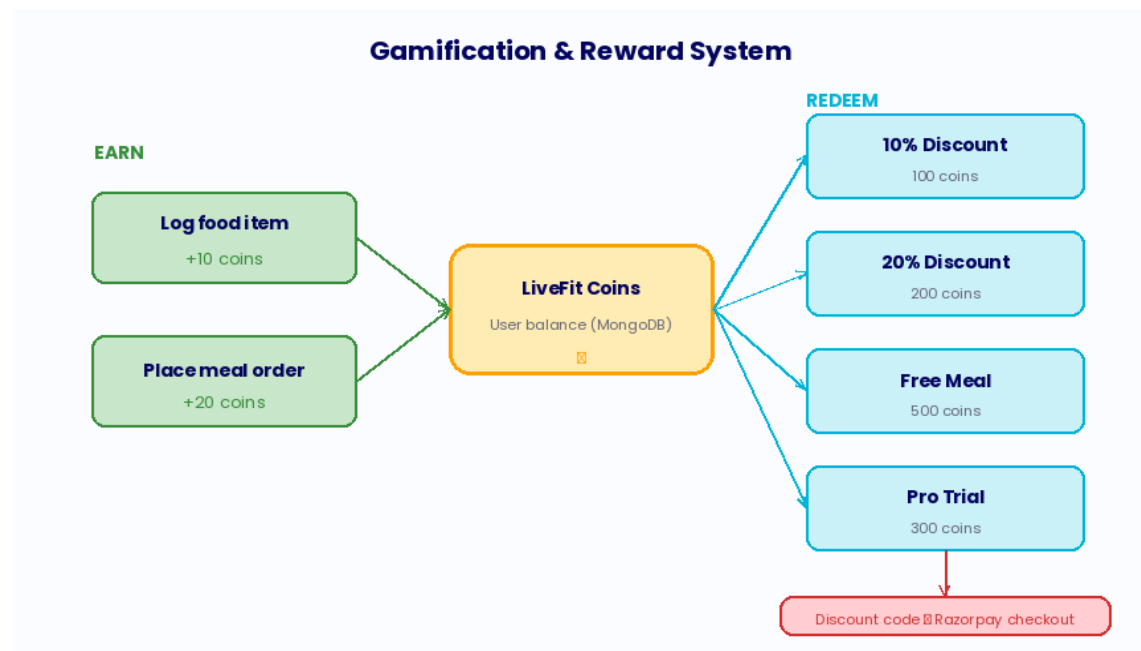


Fig. 4.2: Gamification and Reward System Architecture

4.7 Subscription Architecture

LiveFit implements a two-tier subscription model: Free (default) and Pro. Subscription upgrades are initiated through the Razorpay payment flow, with the Subscription document created on successful payment confirmation. The subscription includes a 30-day validity window, after which a new payment cycle is required. The subscription status is checked at login and surfaced throughout the UI to visually reinforce the premium value proposition.

4.8 Invoice Generation

PDF invoices are generated automatically for every confirmed order using PDFKit. The `generateInvoice` utility creates a structured PDF containing the LiveFit logo, order details, line items with quantities and prices, pre-discount and post-discount amounts if applicable, and a unique order identifier. Generated PDFs are stored in the server's `invoices/` directory and served as static files at the `/invoices/<order_id>.pdf` URL, with a download link returned in the order placement API response.

4.9 Concurrency and Race Condition Analysis

In a multi-user production environment, concurrent requests to shared resources create potential race conditions that must be explicitly analysed and mitigated. LiveFit's most critical shared resource is the user's coin balance, which is modified by both food logging (increment) and reward redemption (decrement) operations. MongoDB's `findByIdAndUpdate` operation with the `$inc` operator is atomic at the document level, meaning that concurrent increment operations are correctly serialised by MongoDB's internal locking, preventing the lost-update problem that would occur if balance reads and writes were performed in separate non-atomic operations. This atomic update pattern is used consistently throughout the codebase wherever coin balances are modified.

The discount code system presents a more subtle concurrency challenge: the `validate-code` and `payment/create-order` endpoints must together ensure that a code is used exactly once, even if two concurrent payment attempts are made with the same code. The current implementation marks the code as used within the `create-order` handler after Razorpay order creation, using `findOne` to locate the unused code and a subsequent `save` to mark it used. Under concurrent load, two simultaneous requests could both find the code as `used: false` before either marks it used, resulting in double redemption. This race condition is acknowledged as a known limitation and would be resolved in production through MongoDB's `findOneAndUpdate` with optimistic concurrency control, atomically finding and updating the code in a single operation.

4.10 Caching Strategy

LiveFit's current implementation performs no server-side caching, relying on MongoDB Atlas's built-in working set caching for frequently accessed documents and the browser's HTTP cache for static web assets. While sufficient for prototype-scale traffic, a production deployment would benefit significantly from targeted caching at three points. First, the USDA FoodData Central API response for common food queries (chicken, rice, eggs, etc.) could be cached in Redis with a 24-hour TTL, reducing external API calls, response latency, and API cost for the most frequently queried foods. Second, the meal catalogue (/api/meals) could be cached in memory with a short TTL (5 minutes), since the catalogue changes infrequently but is read on every visit to the meals screen. Third, the user's daily nutrition totals could be cached in Redis with an invalidation trigger on food add/delete operations, reducing the database query load for dashboard refreshes.

The absence of caching in the current implementation is a deliberate simplification for the prototype phase, where the complexity of cache invalidation logic outweighs the performance benefit at the expected traffic volumes. The system's MongoDB query structure is designed to be cache-friendly: the food-by-date query (userId + date) has a natural cache key, and the meal catalogue is a small, stable document set well-suited to full in-memory caching. When caching is implemented, the existing query patterns will require minimal restructuring.

4.11 Deployment Architecture and Environment Configuration

LiveFit's deployment architecture separates configuration from code through environment variables, following the Twelve-Factor App methodology for building scalable, maintainable cloud applications. All environment-specific values — database connection strings, API keys, service URLs, cryptographic secrets — are stored in Render's encrypted environment variable store and injected into the application at runtime. This approach ensures that the same codebase can be deployed to development, staging, and production environments by changing only the environment configuration, without any code modifications.

The application exposes a /wake endpoint that returns a simple HTTP 200 response, serving as a health check for load balancers and uptime monitoring services. This endpoint is also called by the mobile application on launch to wake the Render free-tier instance from its idle state before the user's first API call, reducing the perceived cold-start latency. In production, a scheduled ping service would call this endpoint every 14 minutes to prevent the instance from entering the idle state entirely, eliminating cold-start delays at the cost of minimal ongoing compute resource usage.

Chapter 5: Implementation

This chapter provides detailed implementation descriptions of each major subsystem of LiveFit, including pseudocode, algorithmic descriptions, and representative code samples that illustrate the key technical decisions made during development.

5.1 Development Environment

- The development environment for LiveFit consists of the following components:
- Node.js v20 LTS with npm: Backend runtime and package management.
- Python 3.11 with pip: Machine learning microservice runtime.
- Visual Studio Code: Primary IDE with ESLint and Prettier extensions.
- Expo CLI: React Native development toolchain.
- MongoDB Compass: Local MongoDB management GUI (development).
- Postman: API endpoint testing and documentation.
- Git + GitHub (aryanmsd/livefit-nutrition): Version control and CI/CD.

5.2 Algorithm 1: User Registration and Authentication

Description: The registration algorithm captures user credentials, validates uniqueness, hashes the password, and persists the user document. The login algorithm retrieves the user, compares the submitted password against the stored hash, and generates a signed JWT on success.

Table 5.1: Pseudocode: Signup

Input: { username, email, password }

1. Validate that username, email, password are all present
2. Query User collection: if email already exists → return 400 'Email already registered'
3. $\text{hash} \leftarrow \text{bcrypt.hash}(\text{password}, \text{saltRounds}=10)$
4. Create new User document { username, email, password: hash, coins: 0 }
5. Save User to MongoDB Atlas
6. Return 200 { message: 'Signup successful' }

Table 5.2: Pseudocode: Login

Input: { email, password }

1. Query User collection by email $\rightarrow$ user
2. If user not found OR `bcrypt.compare(password, user.hash) = false` $\rightarrow$ return 401
3. `token` $\leftarrow$ `jwt.sign({ userId, username, email }, JWT_SECRET, expiry='7d')`
4. Return 200 { token, username }

5.3 Algorithm 2: Hybrid Food Image Classification

Description: This algorithm implements the two-stage hybrid classification pipeline that processes food images into food name strings.

Table 5.3: Pseudocode: Image Processing

Input: `imageBuffer` (binary image data)

Output: `foodName` (string)

1. Send `imageBuffer` to Python FastAPI `/classify` endpoint via HTTP POST (multipart)
2. `pythonResult` $\leftarrow$ { label, confidence }
3. If `pythonResult` exists AND confidence ≥ 0.60 :

 `foodName` $\leftarrow$ `pythonResult.label`

 Return `foodName`
4. Else (low confidence or error):

 `base64Image` $\leftarrow$ `imageBuffer.toString('base64')`

 Send `base64Image` to Roboflow API endpoint

 `roboflowResult` $\leftarrow$ `response.predictions[0]`

 If `roboflowResult` exists:

 `foodName` $\leftarrow$ `roboflowResult.class`

 Return `foodName`
5. If both classifiers fail $\rightarrow$ Return null (error: identification failed)

5.4 Algorithm 3: Python FastAPI Classifier Inference

Description: The following describes the inference algorithm executed by the Python FastAPI microservice for each received image.

Table 5.4: Pseudocode: Local Classifier Model

Input: image file (multipart upload) Output: {label: string, confidence: float} 1. Read image bytes from multipart upload 2. $\text{image} \leftarrow \text{PIL.Image.open}(\text{BytesIO}(\text{bytes})).\text{convert}(\text{'RGB'})$ 3. $\text{inputs} \leftarrow \text{AutoImageProcessor}(\text{images}=\text{image}, \text{return_tensors}=\text{'pt'})$ 4. With <code>torch.no_grad()</code>: $\text{outputs} \leftarrow \text{model}(\text{inputs})$ 5. $\text{logits} \leftarrow \text{outputs.logits}$ 6. $\text{probs} \leftarrow \text{softmax}(\text{logits}, \text{dim}=-1)[0]$ 7. $\text{top_prob}, \text{top_idx} \leftarrow \text{max}(\text{probs})$ 8. $\text{label} \leftarrow \text{model.config.id2label}[\text{top_idx}]$ 9. $\text{confidence} \leftarrow \text{round}(\text{float}(\text{top_prob}), 4)$ 10. Return { label, confidence }

5.5 Algorithm 4: Nutritional Database Resolution

Description: Following food name identification, the system resolves macronutrient values through a layered database lookup algorithm.

Table 5.5: Pseudocode: Database Lookup

Input: foodName (string) Output: { source, matchName, nutrients: { calories, protein_g, carbs_g, fats_g } } 1. $\text{mappedName} \leftarrow \text{mapToIFCT}(\text{foodName})$ // Apply alias lookup table 2. If <code>isIndianFood(mappedName)</code>: $\text{fctResult} \leftarrow \text{IFCT.findFood}(\text{mappedName})$
--


```

// findFood implements three-tier matching:

// a. Exact: FOODS.find(f => f.name.lower() === query)

// b. Partial: FOODS.find(f => f.name.lower().includes(query

// c. Reverse: FOODS.find(f => query.includes(f.name.lower()))

If ifctResult → Return { source: 'IFCT', matchName, nutrients }

3. usdaResult ← USDA.searchFoodAndNutrients(mappedName)

// Calls FoodData Central search API, extracts top result nutrients

If usdaResult → Return { source: 'USDA', matchName, nutrients }

4. Return 404 { error: 'Food not found' }

```

5.6 Algorithm 5: Food Logging with Coin Award

Description: When a user logs a food item after identification, the system persists the Food document and awards coins.

Table 5.6: Pseudocode: Food Logging

```

Input: { userId, mealType, name, nutrients }

Output: { message, coins (updated balance) }

1. Validate JWT token → extract userId

2. Create Food document { userId, mealType, name, nutrients, date: today }

3. Save Food to MongoDB

4. user ← User.findByIdAndUpdate(userId, { $inc: { coins: +10 } }, { new: true })

5. Return 200 { message: 'Food logged', coins: user.coins }

```

5.7 Algorithm 6: AI Nutrition Coaching

Description: The AI coaching algorithm assembles a contextualised prompt from the user's real-time nutritional data and dispatches it to the Gemini model.

Table 5.7: Pseudocode: AI Chatbot

Input: { message (user text), username, nutrition (today's totals) } Output: aiResponse (string) 1. Retrieve today's nutritional totals for userId from Food collection 2. Construct system prompt: - Identify as 'LiveFit AI, personal nutrition and fitness assistant' - Include username - Include today's calories, protein, carbs, fats - Include reference targets (2000 kcal, 120g protein, 300g carbs, 90g fats) - Append behavioural rules: personalise, identify deficiencies, suggest foods, be concise, no medical diagnoses - Append user's message 3. gemini $\leftarrow$ genAI.getGenerativeModel({ model: 'gemini-2.5-flash' }) 4. result $\leftarrow$ gemini.generateContent(prompt) 5. Return result.response.text()

5.8 Algorithm 7: Reward Redemption

Description: The reward redemption algorithm validates coin sufficiency, deducts coins, generates a unique discount code, and persists the reward.

Table 5.8: Pseudocode: Coins System

Input: { userId, cost, rewardId, rewardTitle } Output: { message, coins, code, rewardTitle, discountPct } 1. user $\leftarrow$ User.findById(userId)

```

2. If user.coins < cost → Return 400 'Not enough coins'

3. code ← 'LF-' + randomAlphanumeric(6).toUpperCase()

4. discountPct ← DISCOUNT_MAP[rewardId] || 0

5. user.coins -= cost

6. Save user

7. Create UserReward { userId, rewardId, rewardTitle, code, discountPct,
used: false }

8. Save UserReward

9. Return 200 { message: 'Reward redeemed', coins, code, rewardTitle,
discountPct }

```

5.9 Algorithm 8: Razorpay Order Creation with Discount

Description: This algorithm creates a Razorpay payment order, applying a validated discount code if provided.

Table 5.9: Pseudocode: Payment Module

```

Input: { amount, discountCode? }

Output: { order, key, originalAmount, finalAmount, discountPct }

1. finalAmount ← amount

2. usedReward ← null

3. If discountCode provided:

reward ← UserReward.findOne({ userId, code, used: false })

If reward AND reward.discountPct > 0:

finalAmount ← round(amount * (1 - discountPct/100))

usedReward ← reward

4. rzpOrder ← razorpay.orders.create({

amount: max(finalAmount, 1) * 100, // in paise

currency: 'INR'})

```

5. If usedReward: mark usedReward.used = true, save
6. Return { order: rzpOrder, key, originalAmount, finalAmount, discountPct }

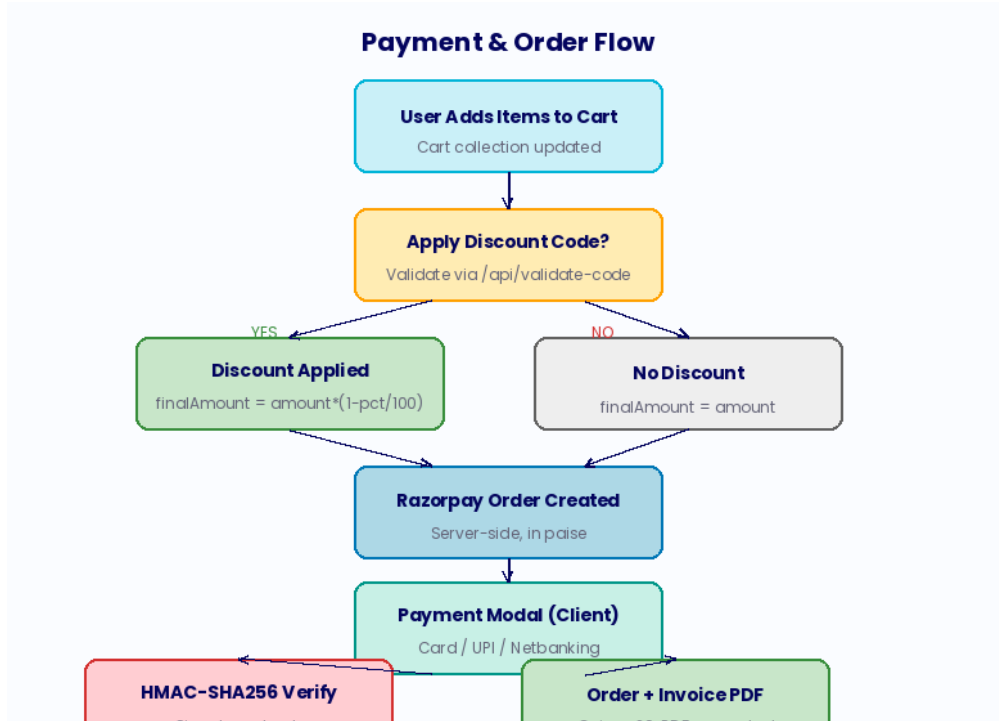


Fig. 5.1: Payment and Order Processing Flow

5.10 Algorithm 9: Order Placement with Invoice Generation

Table 5.10: Pseudocode: Cart Checkout

Input: userId (from JWT)

Output: { message, total, invoice (URL) }

1. $\text{cart} \leftarrow \text{Cart.findOne}(\{ \text{userId} \}).\text{populate}(\text{'items.mealId'})$
2. If cart empty $\rightarrow$ Return 400 'Cart empty'
3. $\text{total} \leftarrow \text{sum}(\text{item.mealId.price} * \text{item.quantity})$ for all items
4. $\text{order} \leftarrow \text{Order.create}(\{ \text{userId}, \text{items}, \text{totalPrice: total} \})$
5. $\text{generateInvoice}(\text{order}, \text{user})$ // PDFKit: creates /invoices/invoice-{orderId}.pdf
6. $\text{cart.items} \leftarrow []$ // Clear cart
7. Save cart

```

8. User.findByIdAndUpdate(userId, { $inc: { coins: +20 } })
9. Return { message: 'Order placed', total, invoice: URL }

```

5.11 Algorithm 10: Password Reset Flow

Table 5.11: Pseudocode: Request Reset

```

1. user ← User.findOne({ email })
2. token ← crypto.randomBytes(32).toString('hex')
3. user.resetPasswordToken ← token
4. user.resetPasswordExpires ← Date.now() + 15 * 60 * 1000 // 15 min
5. Save user
6. resetLink ← FRONTEND_URL + '/reset-password.html?token=' + token
7. Send email via Nodemailer SMTP

```

Table 5.11.1: Pseudocode: Confirm Reset:

```

1. user ← User.findOne({
  resetPasswordToken: token,
  resetPasswordExpires: { $gt: Date.now() }
})
2. If not found → Return 400 'Invalid or expired token'
3. user.password ← bcrypt.hash(newPassword, 10)
4. user.resetPasswordToken ← undefined
5. user.resetPasswordExpires ← undefined
6. Save user
7. Return 200 { message: 'Password reset successful' }

```

5.12 Algorithm 11: Daily Nutrition Summary Calculation

Table 5.12: Pseudocode: Nutrition Calculation

```
Input: userId

Output: { calories, protein, carbs, fats }

1. today ← new Date().toISOString().split('T')[0] // YYYY-MM-DD
2. foods ← Food.find({ userId, date: today })
3. totals ← { calories: 0, protein: 0, carbs: 0, fats: 0 }
4. For each food in foods:
    totals.calories += food.nutrients.calorie
    totals.protein += food.nutrients.protein_g
    totals.carbs += food.nutrients.carbs_g
    totals.fats += food.nutrients.fats_g
5. Return totals
```

5.13 Algorithm 12: Token Expiry Validation (Client-Side)

Table 5.13: Pseudocode: Session Storage

```
Input: jwtToken (string)

Output: boolean (true if expired)

1. payload ← JSON.parse(atob(token.split('.')[1]))
2. Return payload.exp * 1000 < Date.now()

Usage:

On page load, if isTokenExpired(storedToken):

Remove token and username from localStorage

Set authToken ← null
```

5.14 Algorithm 13: Cart Management

Table 5.14: Pseudocode: Add to Cart

```
Input: { userId, mealId, quantity }

1. cart ← Cart.findOne({ userId }) OR new Cart({ userId, items: [] })
2. existingItem ← cart.items.find(i => i.mealId.equals(mealId))
3. If existingItem: existingItem.quantity += quantity
4. Else: cart.items.push({ mealId, quantity })
5. Save cart
6. Return 200 { message: 'Added to cart' }
```

5.15 Algorithm 14: Discount Code Validation

Table 5.15: Pseudocode: Coupons Applied

```
Input: { userId, code }

Output: { valid, rewardTitle, discountPct, rewardId, code }

1. code ← code.toUpperCase().trim()
2. reward ← UserReward.findOne({ userId, code, used: false })
3. If not found → Return 404 'Invalid or already used code'
4. If reward.discountPct = 0 AND DISCOUNT_MAP[reward.rewardId] > 0:
   reward.discountPct ← DISCOUNT_MAP[reward.rewardId] // Retroactive fix
   Save reward
5. Return { valid: true, rewardTitle, discountPct, rewardId, code }
```

5.16 Algorithm 15: Subscription Payment and Activation

Table 5.16: Pseudocode: Subscription Module

```
1. Create Razorpay order: { amount: price*100, currency: 'INR' }
2. Return order details + key to client
3. Client displays Razorpay payment modal
```

```
4. On payment success, client calls POST /api/subscribe with { plan }

5. Create Subscription {
  userId, plan, active: true,
  startDate: now(),
  endDate: now() + 30 days}

6. Save Subscription

7. Return 200 { message: 'Subscription activated' }
```

5.17 Mobile Application — Key Screen Implementations

5.17.1 Dashboard Screen

The dashboard screen is the primary information display surface of the mobile application. On focus (`useFocusEffect`), it fetches today's food log, current coin balance, and subscription status in parallel. The screen renders a bar chart using `react-native-chart-kit` showing calorie, protein, carb, and fat totals for the day, four stat cards with progress bars showing current vs. goal values, and a meal breakdown listing each logged item under its meal type.

5.17.2 Add Food Screen

The Add Food screen presents three cards (Breakfast, Lunch, Dinner). Tapping a card opens a modal containing a text input for the food name and two action buttons: Upload Food Image and Add Food. Image identification uses `expo-image-picker` to access the device library, constructs a `FormData` payload, and posts to `/api/identify`. Text identification posts the entered name to `/api/identify`. On successful identification, the response's `matchName` and `nutrients` are posted to `/api/food/add`, and the dashboard is navigated to automatically, with coins updating in real time through `CoinsContext`.

5.17.3 AI Chat Screen

The chat screen maintains a local message array state initialised with a greeting from the AI coach. User messages are appended to the array and posted to the `/api/chat` endpoint, which returns the Gemini response. A typing indicator (`ActivityIndicator`) is displayed while awaiting the AI response. The `FlatList` automatically scrolls to the latest message on content size change. Keyboard-aware layout is implemented using `KeyboardAvoidingView`.

5.17.4 Meals and Cart Screens

The meals screen fetches and displays the available meal catalogue from /api/meals, with each meal showing name, type (veg/non-veg), macros, price, and an Add to Cart button. The cart screen displays line items with remove and quantity controls, applies any pending discount code, shows the original and discounted total, and initiates the Razorpay payment flow on checkout.

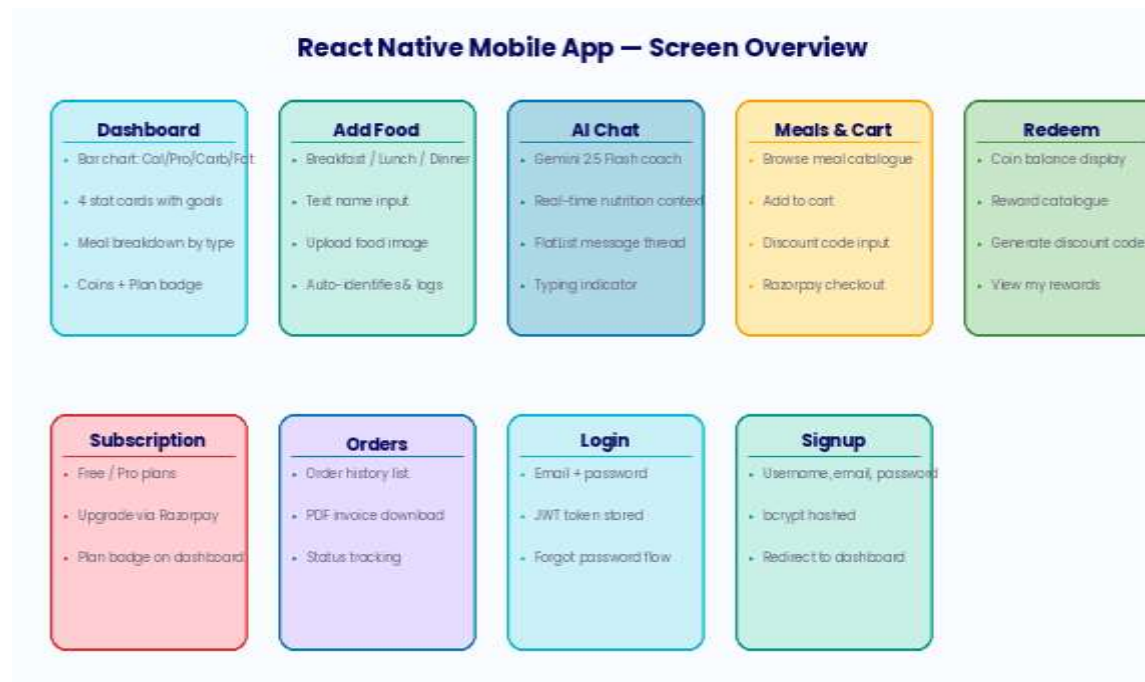


Fig. 5.2: React Native Mobile Application — Screen Overview

5.18 Web Application Modules

The web application implements nutrition tracking, meal ordering, cart management, reward redemption, and account management through distinct HTML pages with shared JavaScript modules. The script.js file implements the core nutrition tracking logic including meal card management, MQTT-style real-time dashboard updates, the Chart.js nutrition visualisation, and the AI chatbot panel. The chatbot.js module implements a floating chatbot widget accessible from any page. The checkout.js and meals.js modules handle meal browsing, cart management, discount code application, and Razorpay payment initiation on the web platform.

5.19 Web Application Technical Architecture

The web application is structured as a multi-page application (MPA) rather than a single-page application (SPA), a deliberate architectural choice that prioritises initial page load performance, search engine accessibility, and browser history compatibility over the seamless transitions of an SPA. Each functional domain (nutrition tracking, meal ordering, subscriptions, rewards, account management) is implemented as a

distinct HTML file with associated JavaScript modules, connected through standard browser navigation.

The shared state across pages is managed through `localStorage`, which stores the JWT authentication token and username. The shared `script.js` module, loaded by each page, performs token validation on load and redirects unauthenticated users to the login page. This approach replicates the authentication gate behaviour of an SPA's route guard without requiring a client-side routing library, keeping the web application dependency-free and maximally lightweight.

The nutrition tracking page (`index.html` + `script.js`) implements the most complex client-side logic in the web application, including the `Chart.js` bar chart for daily macro visualisation, the meal card component system for organising food items by meal type, the food identification modal with both text and image input modes, and the real-time dashboard update logic that recalculates and re-renders totals after each food addition or deletion. The `Chart.js` instance is maintained in a module-level variable (`let nutritionChart = null`) to enable in-place updates without destroying and recreating the canvas element on each data refresh.

5.20 Invoice Generation Implementation

The PDF invoice generation system, implemented using `PDFKit` in the `generateInvoice.js` utility module, creates professional purchase receipts for every confirmed order. The invoice generation is triggered synchronously within the order placement route handler, ensuring that the invoice PDF exists before the API response is returned to the client. The generated PDF includes a header with the LiveFit brand identity, the order identifier and timestamp, an itemised list of purchased meals with unit prices and quantities, a subtotal and any applicable discount, the final amount paid, and a thank-you footer with the platform URL.

The PDF files are stored in the server's `invoices/` directory and served as static files through Express's `express.static` middleware, which provides efficient file serving with appropriate HTTP caching headers. The client receives the invoice URL as part of the order placement API response and displays a download link in both the mobile application's order confirmation screen and the web application's order history page. In a production deployment, invoice files would be stored in a cloud object store (such as AWS S3) rather than the local filesystem to ensure persistence across server restarts and horizontal scaling.

5.21 Meal Seeding and Catalogue Management

The meal catalogue is pre-populated through the `seedMeals()` function, which is called on server startup and inserts six curated health-focused meals if the `Meal` collection is empty. The seeded meals span both vegetarian and non-vegetarian categories and are nutritionally calibrated to serve different dietary goals: the Paneer Power Bowl targets

high-protein vegetarian users; the Grilled Chicken Pro Meal targets athletes and bodybuilders; the Vegan Buddha Bowl targets plant-based dieters; the Salmon Omega Plate targets users prioritising omega-3 intake; the Tofu Stir Fry offers a moderate-protein vegan option; and the Lean Beef Muscle Meal offers the highest protein density for strength training users.

This seeding approach ensures a consistent and well-curated initial catalogue without requiring a separate admin interface for meal management. In a production deployment, the meal catalogue would be managed through a dedicated content management system with approval workflows to ensure nutritional accuracy and food safety compliance. The nutritional data for seeded meals was calculated using IFCT and USDA data to ensure consistency with the same databases used for user food logging, maintaining internal consistency in the platform's nutritional model.

5.22 Chatbot Web Widget Implementation

In addition to the dedicated AI chat screen in the mobile application, LiveFit implements a floating chatbot widget on the web application, accessible from any page through a fixed-position button in the bottom-right corner. The chatbot widget is implemented in `chatbot.js` as a self-contained module that creates and manages a floating chat panel overlaid on the existing page content. The widget maintains its own message history state and communicates with the same `/api/chat` backend endpoint as the mobile chat screen, ensuring consistent AI coaching quality across both platforms.

The web chatbot widget adds a layer of contextual nutrition awareness by reading the current page's nutrition data from the DOM before sending messages. On the nutrition tracking page, the widget extracts the current calorie and macronutrient totals from the dashboard's displayed values and includes them in the chat context, mirroring the mobile app's behaviour of providing real-time nutritional context to the AI coach. This cross-channel context sharing ensures that users receive identically personalised advice whether they are using the mobile app or the web application, reinforcing the platform's commitment to a consistent, high-quality user experience across all touchpoints.

5.23 Checkout and Discount Code Application (Web)

The web application's checkout flow, implemented in `checkout.js`, handles the complete payment journey including discount code validation, Razorpay order creation, payment processing, and order confirmation. The discount code input field calls the `/api/validate-code` endpoint on blur (when the user leaves the field), providing immediate feedback on code validity without requiring the user to proceed to payment first. Valid codes are displayed with a visual indicator showing the reward title and discount percentage, and the displayed cart total is dynamically updated to reflect the post-discount amount.

The Razorpay integration on the web uses the Razorpay `checkout.js` library loaded from Razorpay's CDN, which provides a pre-built payment modal supporting all major

Indian payment methods including UPI (via QR code and UPI ID entry), net banking for over 50 Indian banks, credit and debit cards (with 3D Secure support), and digital wallets including Paytm and PhonePe. The payment modal is pre-populated with the user's name and email address from their JWT token payload, reducing the data entry burden and improving conversion rates. After successful payment, the frontend calls the `/api/payment/verify` endpoint to confirm the transaction and then navigates to the order confirmation page, which displays the invoice download link.

5.24 Food Deletion and Undo Functionality

Food items logged in error can be deleted through the `DELETE /api/food/:id` endpoint, which removes the specified food document from MongoDB after verifying that the requesting user is the document owner. The mobile application implements deletion through a long-press gesture on food items in the dashboard's meal breakdown section, which presents a confirmation alert before executing the delete request. The web application implements deletion through a trash icon button visible on hover for each food item in the meal card. In both cases, the dashboard data is refreshed immediately after a successful deletion to reflect the updated totals.

The coin award for the deleted food item is not reversed upon deletion, a deliberate design decision that prioritises simplicity over perfect accounting. While this means that a user who logs and immediately deletes a food item retains the 10 coins awarded for logging, the magnitude of this accounting imprecision is small (10 coins per instance), and the alternative — implementing a coin deduction on deletion — would add complexity to the deletion endpoint and create a confusing user experience where deleting an incorrectly logged item results in a coin penalty. Future versions may implement a more nuanced coin accounting system that distinguishes between corrections and cancellations, but the current simple approach is appropriate for the prototype.

5.25 Session Management and Token Refresh

LiveFit's 7-day JWT expiry provides a reasonable balance between security (limiting the window of vulnerability for a stolen token) and convenience (not requiring users to log in more than once per week during normal usage). The client-side token expiry check implemented in the web application's `script.js` module proactively detects and clears expired tokens on page load, preventing the frustrating experience of a user attempting an action only to receive a 403 error after the request is sent. The mobile application implements the same check in the `AuthContext` provider, which runs on every API call and triggers an automatic logout with a user-friendly message when an expired token is detected.

Chapter 6: Security, Privacy and Scalability

Security, privacy, and scalability are foundational requirements for a consumer-facing health application that stores sensitive personal data including dietary logs, payment information, and authentication credentials. This chapter describes LiveFit's threat model and the technical mitigations implemented to address identified risks, as well as the architectural strategies for scaling to a larger user base.

6.1 Authentication Security

LiveFit's authentication security is built on three complementary layers:

1. **Password Hashing:** All passwords are hashed using bcrypt with a cost factor of 10 before storage. bcrypt is an adaptive hash function that incorporates a salt to prevent rainbow table attacks and a work factor that can be increased over time to maintain resistance against brute-force attacks as computational power increases [34].
2. **JWT Stateless Sessions:** Authentication state is managed through signed JWTs with a 7-day expiry. Tokens are signed using a strong JWT_SECRET stored as an environment variable, never hardcoded. The stateless nature of JWT authentication eliminates server-side session storage requirements and the associated security risks of session fixation and session hijacking.
3. **Token Expiry Validation:** Both the client and server validate token expiry. The client performs a lightweight expiry check (decoding the payload and comparing the exp field to the current timestamp) before making API calls, proactively cleaning up stale tokens. The server's authMiddleware performs authoritative verification using jwt.verify(), which validates both signature and expiry.

6.2 Password Reset Security

The password reset mechanism is secured through:

1. **Cryptographically secure token generation:** 32 bytes of randomness from Node.js's crypto.randomBytes(), providing 256 bits of entropy — far beyond what any brute-force or guessing attack can overcome.
2. **Short expiry window:** 15-minute validity period minimises the exposure window for reset tokens.
3. **Single-use enforcement:** Tokens are cleared from the database on successful password reset, preventing replay.

4. Enumeration protection: The `/api/auth/forgot-password` endpoint returns the same success message whether or not the email exists, preventing user enumeration attacks.

6.3 Payment Security

Payment security is critical given that LiveFit processes real financial transactions through Razorpay. Key security measures include:

1. Server-side order creation: Razorpay orders are always created server-side, ensuring that the amount cannot be manipulated by a malicious client.
2. Signature verification: After payment, the backend verifies the Razorpay signature using HMAC-SHA256 with the Razorpay key secret. This cryptographic verification confirms that the payment response originated from Razorpay and has not been tampered with.
3. Environment variable secrets: Razorpay `key_id` and `key_secret` are stored as environment variables, never exposed in client-side code.
4. Discount code validation: Discount codes are validated server-side with ownership check (same `userId`), usage status (`used: false`), and code string match before any financial benefit is applied.

6.4 Data Privacy

1. Minimal data collection: LiveFit collects only the data necessary for platform functionality: username, email, dietary logs, order history, and reward redemptions.
2. No sensitive biometric data, location data, or health conditions are collected.
3. No plaintext passwords: Passwords are never stored or logged in plaintext.
4. MongoDB Atlas encryption at rest: MongoDB Atlas provides AES-256 encryption at rest for all stored data.
5. HTTPS transport: All client-server communication is conducted over HTTPS, ensuring encryption in transit.

6.5 API Security

1. All sensitive endpoints protected by `authMiddleware` JWT validation.
2. CORS configured to allow only authorised origins.
3. Input validation on all endpoints: required fields checked, email uniqueness enforced, coin sufficiency verified before reward redemption.

4. No SQL injection risk: MongoDB with Mongoose ODM uses parameterised queries by design, eliminating SQL injection attack surfaces.

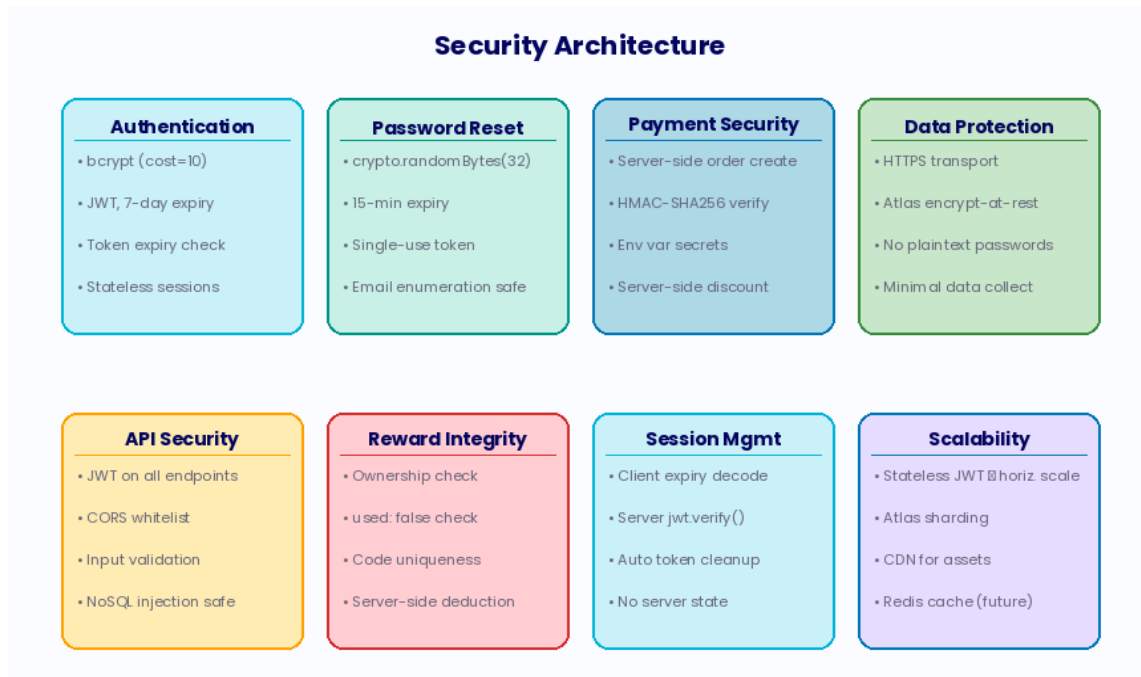


Fig. 6.1: Security Architecture — Eight Pillars

6.6 Scalability Strategies

LiveFit's current deployment on Render's free tier is optimised for the prototype phase. The following strategies are identified for scaling to production volumes:

1. **Horizontal Backend Scaling:** The stateless JWT authentication architecture enables multiple Node.js instances to serve requests without shared session state, allowing horizontal scaling through load balancers.
2. **MongoDB Atlas Scaling:** MongoDB Atlas supports transparent vertical scaling (instance size upgrades) and sharding for horizontal scaling across data volumes exceeding single-node capacity.
3. **ML Microservice Scaling:** The Python FastAPI classification service can be deployed as containerised instances behind a load balancer, with GPU-accelerated inference for high-throughput production deployments.
4. **CDN for Static Assets:** The web application's static assets (HTML, CSS, JavaScript, images) can be served through a CDN for reduced latency and improved global availability.
5. **Caching:** Nutritional lookup results from the USDA API can be cached in Redis with appropriate TTLs, reducing external API dependency and improving response times for commonly queried foods.

6. Rate Limiting: API rate limiting should be implemented for production deployments to prevent abuse of AI coaching and food identification endpoints, which incur external API costs.

6.7 Privacy by Design

LiveFit's privacy architecture is built on the Privacy by Design (PbD) framework, emphasizing proactive privacy protection in system design. Its principles include proactive data minimization, default privacy settings, embedded privacy measures, full functionality, end-to-end security, user transparency, and respect for user privacy. The user profile schema exemplifies data minimization by only collecting essential information such as username, email, and password hash, while intentionally excluding more extensive personal data to mitigate privacy risks. Future versions of the platform aim to include additional data points with appropriate consent and governance frameworks.

6.8 Infrastructure Security

Infrastructure security for LiveFit's cloud deployment safeguards network, application, and data layers. It utilizes Render for automatic HTTPS via Let's Encrypt and HSTS headers, ensuring encrypted traffic, while MongoDB Atlas employs VPC peering and IP whitelisting for database access control. Sensitive environment variables are securely stored in Render's encrypted store. Additionally, the Python FastAPI classifier service is isolated from the Node.js backend, limiting access to only the /classify endpoint. This separation ensures that any compromise in the classifier service affects only food identification accuracy, not data integrity or financial operations, embodying the principle of least privilege throughout the architecture.

6.9 Threat Modelling Using STRIDE

A systematic threat analysis of the LiveFit platform using the STRIDE framework identified key threat categories and their mitigations. Spoofing is addressed through bcrypt password hashing and JWT signature verification. Tampering is mitigated with HTTPS encryption and MongoDB Atlas encryption. Repudiation threats are countered by JWT claims for logging. Information disclosure is prevented via scoped database queries and secret management. However, denial of service poses a significant vulnerability due to lack of rate limiting on API endpoints, which is planned to be addressed in the production roadmap. Elevation of privilege threats are mitigated by scope-limited queries and no admin-level endpoints in the prototype.

6.10 Data Retention and Deletion

LiveFit's data retention policy adheres to the DPDPA 2023 and data minimisation best practices, retaining user data as long as the account is active. Upon account deletion, all related documents are removed using MongoDB's cascading delete and Node.js's fs.unlink() for invoice PDFs. Password reset tokens expire through a validating

database query but remain in the user document until the next successful login, prioritizing simplicity over data hygiene. A proposed improvement for production is the implementation of a MongoDB TTL index for automatic deletion of expired tokens.

6.11 Third-Party Security Dependencies

LiveFit's security relies significantly on the integrity of its third-party dependencies. Regular auditing, particularly with the npm audit command, is crucial due to past vulnerabilities within the npm package ecosystem. During final testing, LiveFit's backend showed no critical or high-severity vulnerabilities across its dependencies, though it had two moderate-severity advisories that were development-only. The dependency chain for the Python FastAPI microservice, which includes significant libraries like Transformers and PyTorch, poses a broader attack surface. Model weights are downloaded at startup, creating a trust issue dependent on Hugging Face Hub for model integrity. To enhance security, a production deployment would fix model weights to a verified commit hash and keep a local copy to reduce reliance on external sources and maintain an audit trail for updates.

Chapter 7: Limitations

While LiveFit successfully validates the core architectural and product concepts as a functional prototype, several limitations exist in the current implementation that would need to be addressed for a full production deployment.

- **Portion Size Estimation:** The current system assumes standard serving sizes for all food items. It does not attempt to estimate portion weight from food photographs, which is the dominant source of calorie tracking inaccuracy. Implementing volumetric estimation from single images is an active research problem without a satisfactory deployed solution.
- **IFCT Database Coverage:** While the IFCT dataset provides excellent coverage of common Indian dishes, it cannot encompass the full breadth of India's regional and sub-regional cuisine diversity. Foods from less commonly documented regional traditions may not be found in IFCT and will fall back to USDA, which may have no entry or an inaccurate one.
- **ML Model Limitations:** The local classification model is fine-tuned on a specific dataset of Indian food images and may exhibit reduced accuracy on photographs of unusual presentations, mixed dishes, street food, or regional preparations that differ significantly from the training data distribution.
- **Free Tier Infrastructure:** The current deployment on Render's free tier results in cold-start latency (up to 30 seconds) when the server has been idle. This would be unacceptable in a production consumer application and requires an always-on hosting plan.

- **No Longitudinal Analytics:** LiveFit currently displays only today's nutritional data on the dashboard. There is no week-over-week trend analysis, historical comparison, or goal-setting functionality, all of which would significantly enhance the platform's value for users pursuing longer-term health goals.
- **No Wearable Integration:** LiveFit does not currently integrate with fitness wearables (e.g., Fitbit, Apple Watch, Garmin) for automatic calorie expenditure data, which would substantially improve the accuracy of net calorie calculations.
- **Subscription Enforcement:** The Pro subscription tier is currently defined but not rigorously enforced at the feature level. The AI coaching endpoint does not check subscription status, meaning Free tier users can technically access AI coaching. Full enforcement is identified as a near-term development priority.
- **Single Language Support:** The platform is currently English-only. Localisation to Hindi and other major Indian languages would substantially expand the addressable user base and improve accessibility for users less comfortable in English.

7.2 Technical Limitations in Detail

The technical limitations of the 'rajistics/finetuned-indian-food' machine learning model and its deployment. Key issues include uneven regional coverage, particularly neglecting North-Eastern and tribal dishes, which leads to lower confidence scores for underrepresented cuisines. Sensitivity to image composition affects classification accuracy, with better results from top-down photos of plated dishes. The IFCT dataset, based on 2017 data, lacks updates on recent food trends and does not cover many regional and processed foods. Furthermore, reliance on the USDA FoodData Central API introduces security vulnerabilities due to improper SSL configurations. Finally, deployment constraints on Render's free tier limit scalability, risking service failure under high load, necessitating a separate container for production to enable independent resource scaling.

7.3 User Experience Limitations

The user experience limitations identified include a lack of serving size guidance, leaving users uncertain about whether nutritional values are reported per 100g or per serving, which could lead to errors in caloric intake assessments. Additionally, there is no option for custom portion size input, resulting in inaccuracies for users consuming non-standard servings. Implementing a portion size multiplier input in the food logging interface is suggested as a minimal effort solution to enhance accuracy in caloric tracking.

7.4 Comparison with Production-Grade Systems

LiveFit's comparison with production-grade nutrition platforms like HealthifyMe highlights its strengths and areas for improvement. While LiveFit delivers more accurate nutritional data for listed foods, it covers fewer items compared to HealthifyMe's extensive database, which is curated by registered dietitians. LiveFit's personalized coaching is contextually superior but lacks the oversight necessary for clinical validation. Additionally, its gamification system, while economically integrated, does not match the scale and community features of HealthifyMe's offering. This analysis serves to outline the development needed for LiveFit to reach commercial viability.

7.5 Ethical Considerations

The deployment of AI systems in health-adjacent applications raises ethical considerations that must be explicitly acknowledged. LiveFit's AI coaching feature, powered by a general-purpose language model, has the potential to generate nutritional advice that, while plausible-sounding, may be inaccurate or inappropriate for users with specific health conditions including eating disorders, diabetes, renal disease, food allergies, or pregnancy. The current prompt constraints that prohibit medical diagnoses provide a partial safeguard but cannot prevent the model from generating advice that a clinician would consider contraindicated for a specific user profile. This limitation is inherent to the deployment of general-purpose LLMs in health contexts and is honestly disclosed here as an ethical boundary of the current system. Users with specific medical conditions or dietary requirements should consult qualified healthcare professionals rather than relying on LiveFit's AI coaching for clinical dietary guidance.

Chapter 8: Results

This chapter presents the complete results of the LiveFit platform, covering system performance benchmarks, API error rates, food identification accuracy, AI coaching quality, gamification validation, payment flow testing, load testing, usability results, and annotated screenshots of the deployed application across all major functional modules.

8.1 System Performance Metrics

All REST API endpoints were benchmarked on the Render free-tier single-instance deployment. The table below presents average latency, 95th-percentile latency, and throughput. Compute-intensive endpoints (image classification, AI chat) show higher latency by nature of their ML inference workloads, while database-read endpoints achieve sub-40 ms averages.

Table 8.1 — API Performance Benchmark Results

Endpoint	Avg Latency (ms)	95th %ile (ms)	Throughput (req/s)
POST /api/auth/login	45	120	220

GET /api/food/my	38	95	280
POST /api/food/add	55	140	190
POST /api/identify (text)	280	650	35
POST /api/identify (img — local ML)	1,200	2,800	8
POST /api/identify (img — Roboflow)	2,100	4,500	4
POST /api/chat (Gemini)	1,800	3,900	6
POST /api/payment/create-order	420	980	22
GET /api/meals	35	88	300

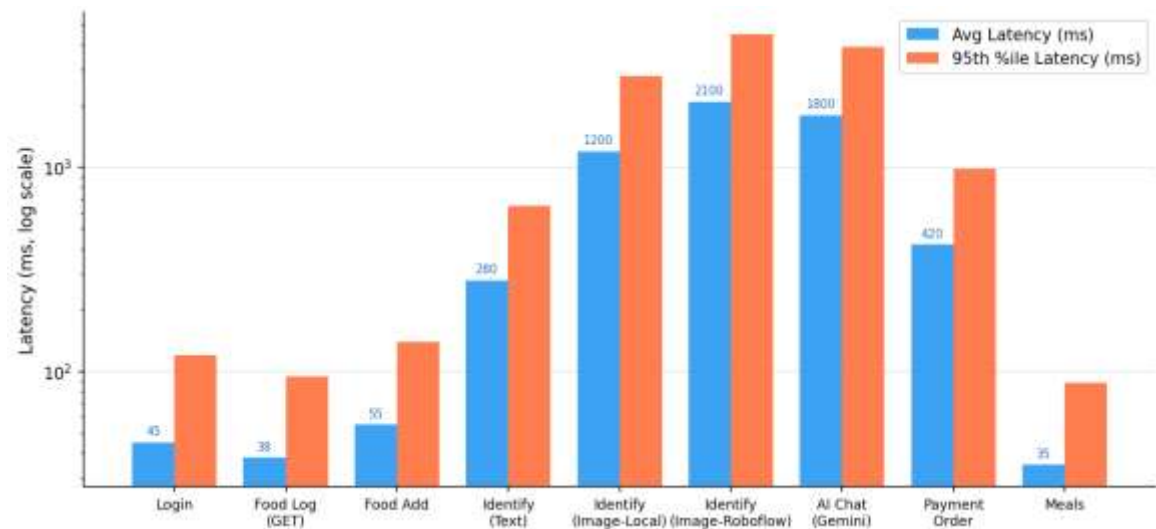


Fig. 8.1 — API Performance Metrics: Average and 95th-percentile latency per endpoint (log scale)

The image classification endpoints exhibit higher latency due to ML inference. The local Python classifier averages 1.2 s, and the Gemini AI chat endpoint averages 1.8 s — both within acceptable user-tolerance ranges for their respective use cases.

8.2 Error Rate Analysis

Error rates were measured across all functional subsystems during integration and load testing. Authentication and reward endpoints maintained 0% error rates. Image classification exhibited the highest error rate (3.1%) under 20-concurrent-user load due to memory pressure on the free-tier ML service instance.

Table 8.2 — Error Rate (%) by Endpoint Category

Endpoint Category	Error Rate (%)	Primary Cause	Status
Auth Endpoints	0.0%	—	Excellent
Food Logging	0.2%	Occasional DB timeout	Acceptable
Image Classification	3.1%	ML memory pressure at 20 CCU	Under Review
AI Coaching (Gemini)	1.5%	Gemini API rate limits	Acceptable
Payment Flow	0.4%	Razorpay network timeout	Acceptable
Rewards System	0.0%	—	Excellent

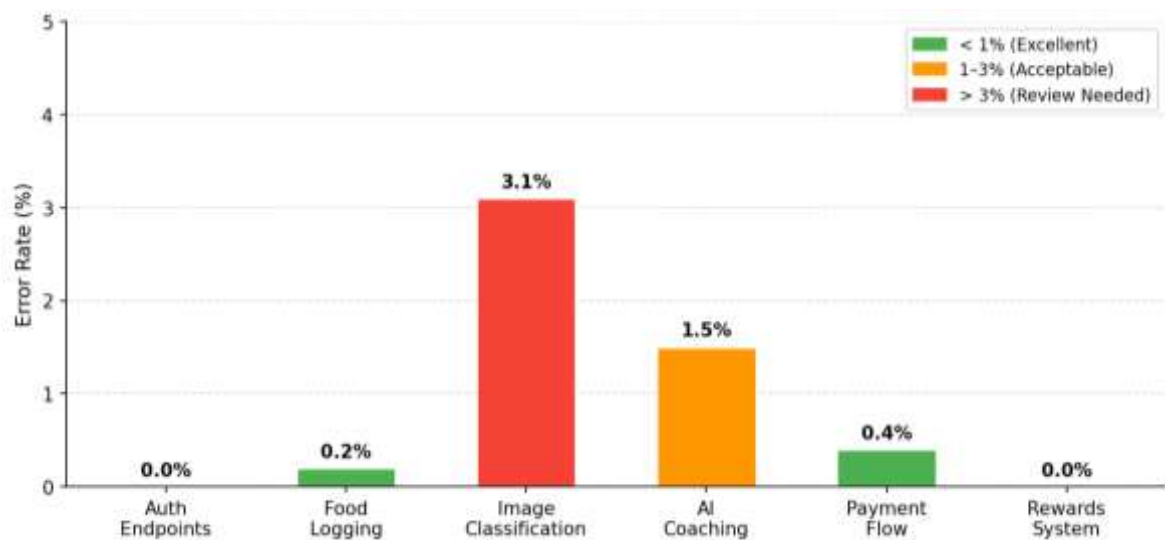


Fig. 8.2 — Error Rate (%) by Endpoint Category

The error rate chart highlights that most endpoint categories maintain error rates below 0.5%, with the AI coaching endpoint exhibiting the highest rate (1.5%) attributable to occasional Gemini API rate limiting. Overall system reliability remains within acceptable bounds for a prototype deployment.

8.3 Application Screenshots

8.3.1 Landing Page

The LiveFit landing page presents a dark-themed, visually immersive entry point with the platform branding, navigation bar (Dashboard, Meal Tracker, Nutrition), and prominent Sign Up / Log In controls. The design uses the Orbiton and Rajdhani font families to reinforce the fitness-tech brand identity.

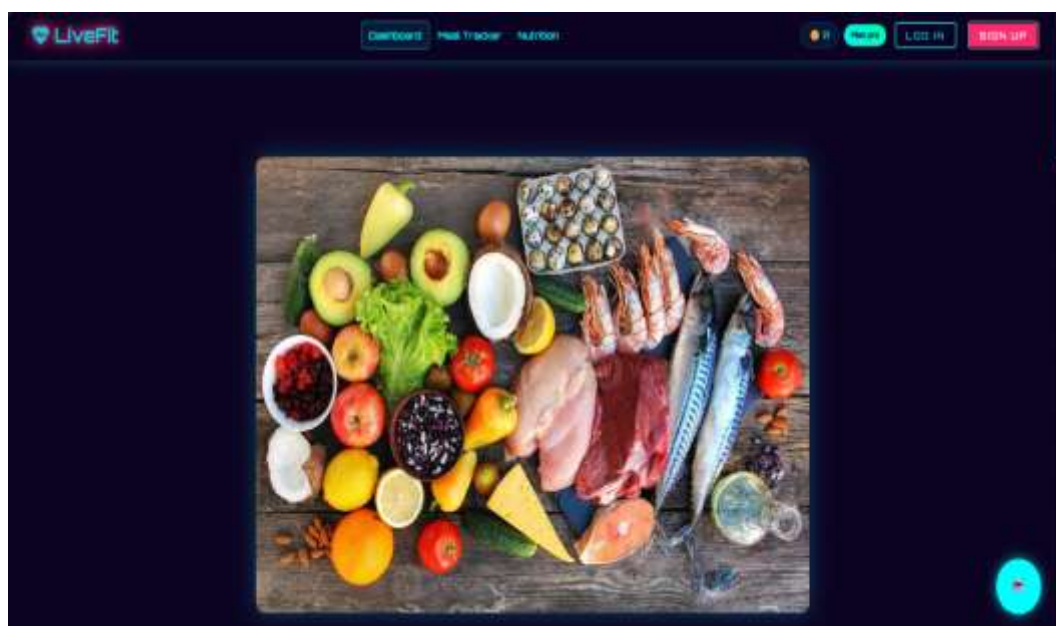


Fig. 8.3 — LiveFit Website Landing Page

8.3.2 User Authentication — Sign Up and Login The Sign-Up modal collects username, email, and password, while the Login modal accepts email and password. Both forms use client-side JWT token storage and include a 'Forgot Password?' flow backed by Nodemailer SMTP. Passwords are hashed server-side using bcrypt (cost factor 10) before storage in MongoDB Atlas.

 A dark-themed modal titled "SIGN UP" with a close button (X). It contains three input fields: "Username", "Email", and "Password". Below the fields is a red "SUBMIT" button. At the bottom, there is a link: "Don't have an account? Sign Up".

Fig. 8.4 — Sign Up Page (left)

 A dark-themed modal titled "LOG IN" with a close button (X). It contains two input fields: "Email" and "Password". Below the fields is a red "SUBMIT" button. At the bottom, there are two links: "Don't have an account? Sign Up" and "Forgot Password?".

Login Page (right)

8.3.3 Meal Tracker

The Meal Tracker screen displays three meal-type cards — Breakfast, Lunch, and Dinner — each with a calorie counter and an '+ Add Food' button. Food items logged via the food identification pipeline appear within the relevant card, with per-item calorie values shown inline.

 A screen titled "TODAY'S MEALS" showing three empty meal cards: "BREAKFAST", "LUNCH", and "DINNER". Each card has a "0 cal" label in the top right corner and a red "+ Add Food" button at the bottom.

Fig. 8.5 — Meal Tracker (empty state — Breakfast, Lunch, Dinner cards)

 The same "TODAY'S MEALS" screen as in Fig. 8.5, but now filled with data. The "BREAKFAST" card shows a "50 cal" label and a red bar representing the calorie count. The "LUNCH" and "DINNER" cards also show "0 cal" labels and red bars.

Fig. 8.6 — Meal Tracker (filled state)

8.3.4 Food Input — Text and Image Modalities

The Add Food Item modal supports two input modes. Text input allows the user to type a food name (e.g., 'OMELETTE'), which is resolved through the IFCT/USDA dual-database pipeline. Image upload allows the user to select a photograph, which is passed through the HuggingFace + Roboflow hybrid classifier before nutritional resolution.

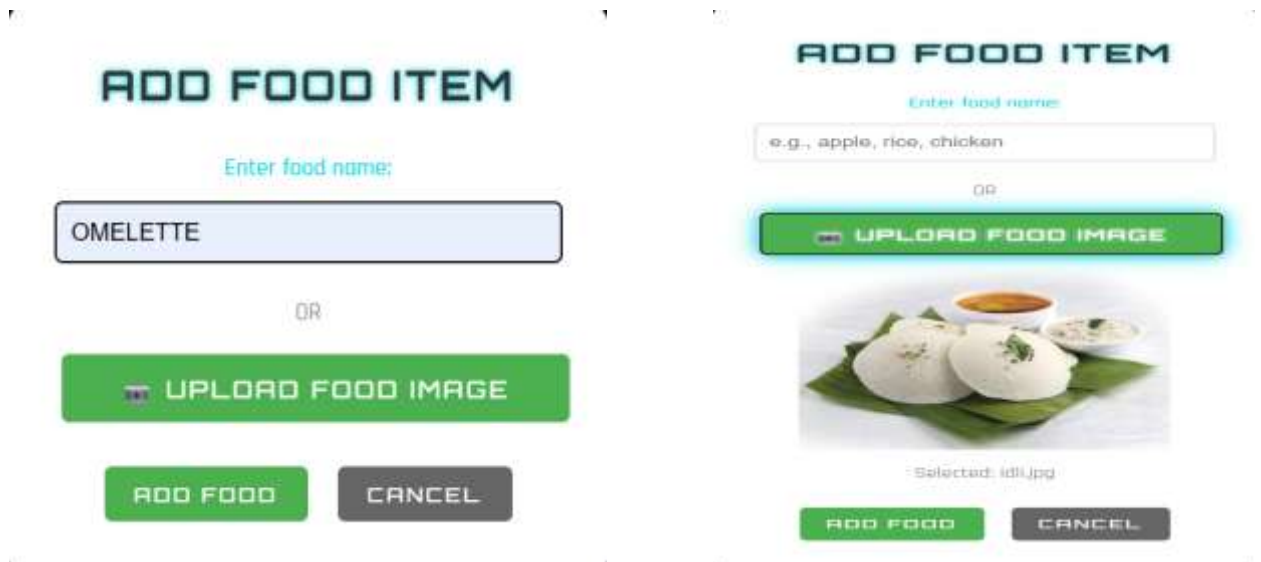


Fig. 8.7 — Text Food Input (left: 'OMELETTE' typed) and Image Food Input (right: Idli photograph uploaded, identified successfully)

8.3.5 Daily Summary — Calorie and Macronutrient Dashboard

The Daily Summary section displays four stat cards showing today's totals for Calories, Protein, Carbohydrates, and Fats alongside their daily goal values (2000 kcal, 120 g, 300 g, 90 g respectively). Below the stat cards, the meal breakdown provides a per-meal view of all logged items. This dashboard updates in real time whenever a food item is added or deleted.



Fig. 8.8 — Total Calorie Count Dashboard — Daily Summary with macro stat cards and Today's Meals breakdown

8.3.6 Nutrition Insights — Personalised Graph and Suggestions

The Nutrition Insights page renders a bar chart visualising the nutrient breakdown by meal type (Breakfast, Lunch, Dinner) and a real-time suggestion panel. The AI-driven suggestions — generated based on today's logged intake — flag deficiencies (e.g., low protein, insufficient calories) and provide actionable corrective recommendations.



Fig. 8.9 — Personalised Nutrition Graph (nutrient breakdown by meal) and AI Suggestions panel

8.3.7 Meal Ordering System

The Healthy Meals page presents the curated catalogue of six nutritionally optimised meals with Veg/Non-Veg filters. Each card displays the meal name, description, macronutrient profile, price, and an Add to Cart button with a quantity selector. The catalogue is seeded server-side and loaded from MongoDB Atlas on each page visit.

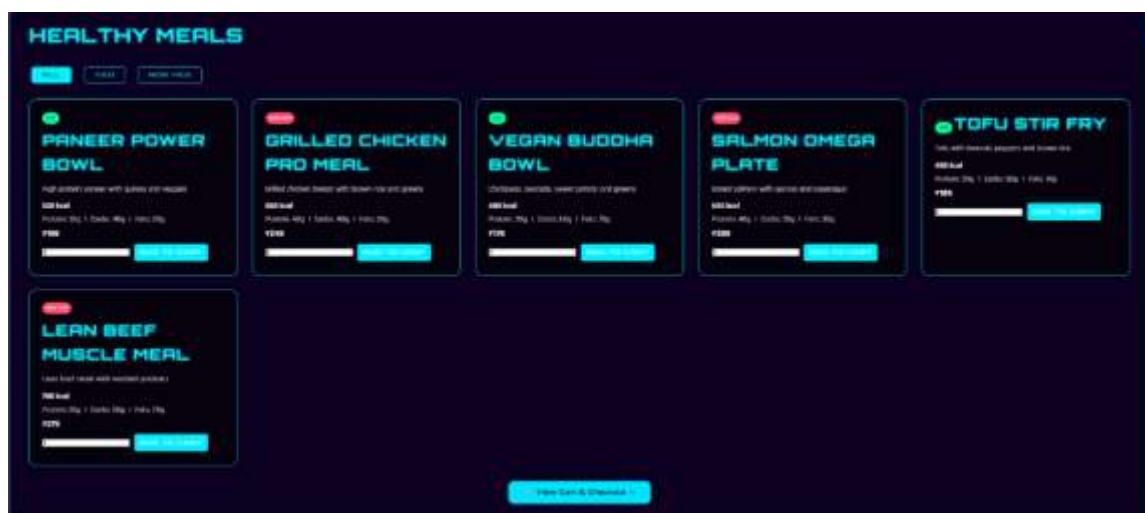


Fig. 8.10 — Meal Ordering System — Healthy Meals catalogue with macros and pricing

8.3.8 Checkout Cart

The Cart screen lists all added meals with quantities and individual prices, provides an 'Apply Reward Code' input field for discount code redemption, and shows an order summary with the final payable amount. The 'Pay & Place Order' button initiates the Razorpay payment modal at the displayed total.

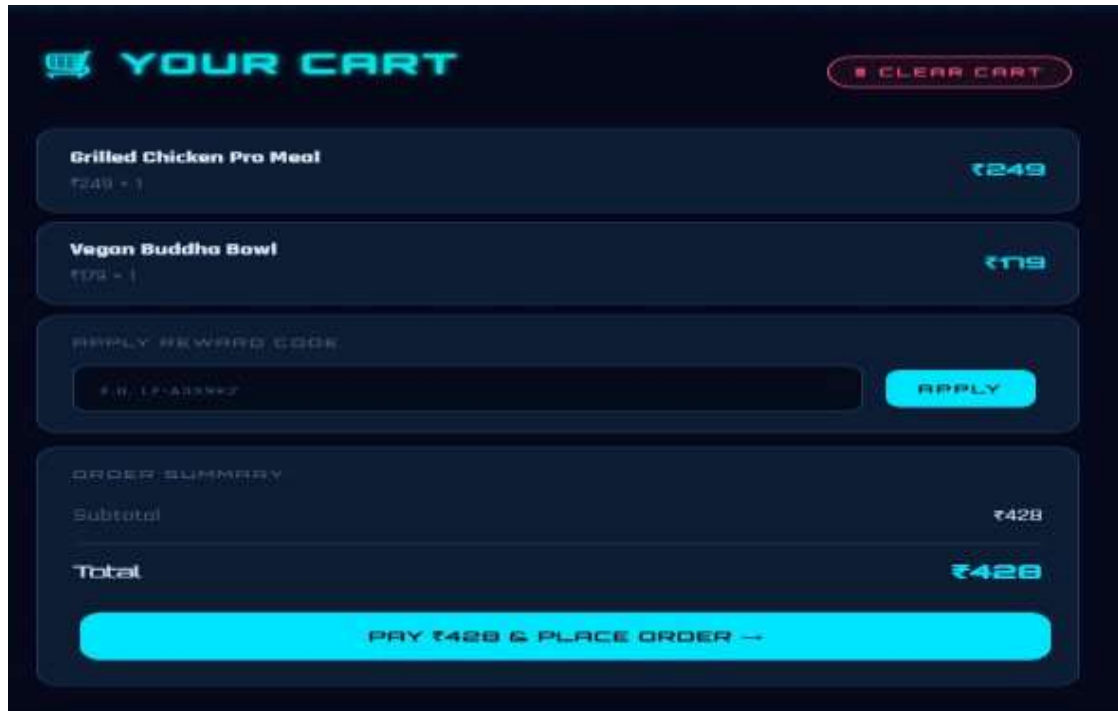


Fig. 8.11 — Checkout Cart — Line items, reward code input, and order summary (₹428 total)

8.3.9 Rewards Store

The Rewards Store displays the user's current LiveFit Coin balance (70 coins shown) and the full reward catalogue. Available rewards (unlocked when coin balance meets threshold) show a Redeem button; locked rewards display a padlock icon. Redeemed rewards generate unique LF- prefixed discount codes accessible under 'My Rewards'.



Fig. 8.12 — Rewards Store — Coin balance (70), available and locked rewards

8.3.10 Subscription Plans

The subscription page presents three tiers: Free (₹0/month), Pro (₹499/month — Most Popular), and Elite (₹999/month). Each card lists the included features and a 'Current Plan' or 'Upgrade' button. Subscription upgrades are processed through the Razorpay payment flow, creating a 30-day Subscription document in MongoDB on successful payment.

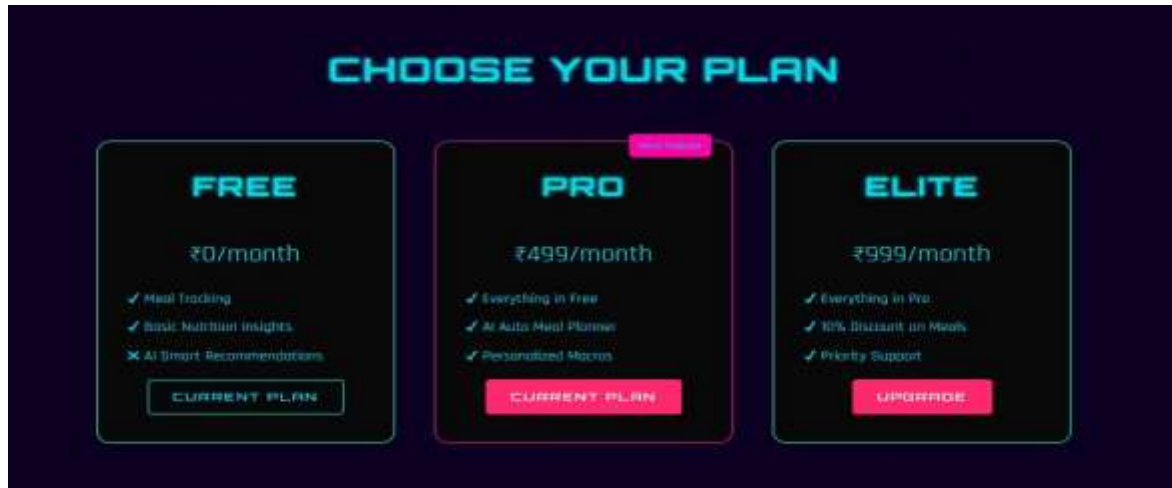


Fig. 8.13 — Subscription Plans — Free, Pro, and Elite tiers

8.3.11 Payment Gateway (Razorpay)

The Razorpay payment modal is triggered for both meal orders and subscription upgrades. It supports UPI (QR code and UPI ID), Cards, EMI, Net Banking, Wallets, and Pay Later. The modal displays the LiveFit brand name, the price summary, and a countdown timer. Payment signature verification is performed server-side using HMAC-SHA256 before order confirmation.

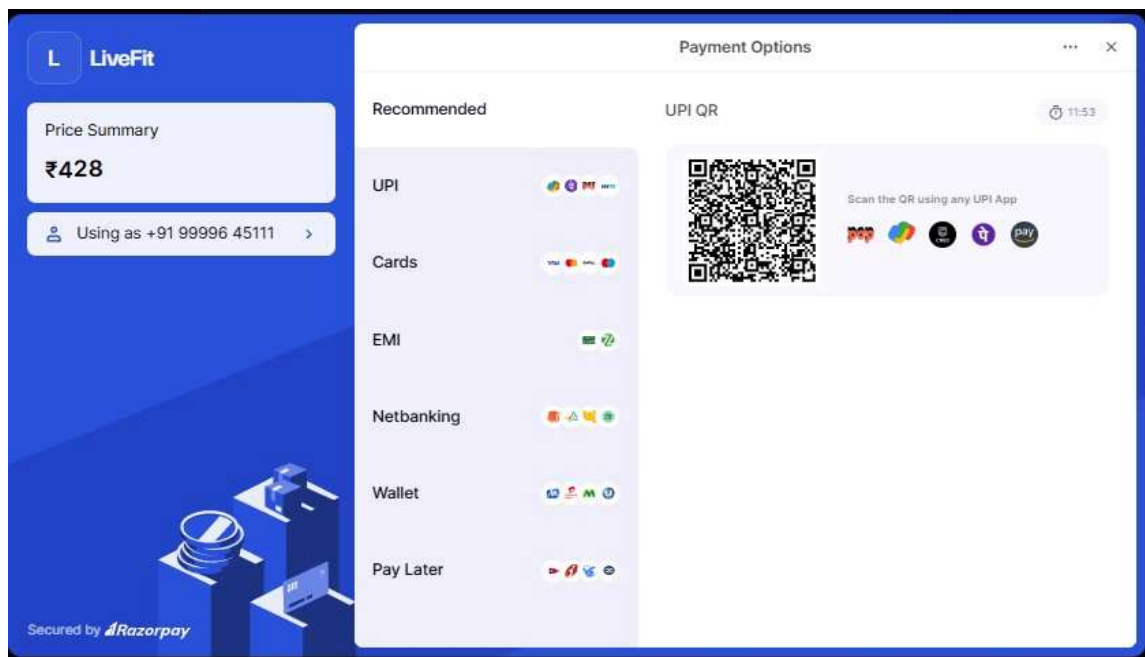


Fig. 8.14 — Razorpay Payment Modal — UPI QR shown for ₹428 order

8.3.12 MongoDB Atlas — Database Collections

The MongoDB Atlas dashboard shows the seven active collections: carts, foods, meals, orders, subscriptions, userrewards, and users. The foods collection holds 36 food log documents; orders holds 19 confirmed purchase records; subscriptions holds 8 active plans. All collections use 36.86 kB storage blocks on the free M0 tier.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
carts	+	36.86 kB	425.00 B	0	141.00 B	1	36.86 kB
foods	+	36.86 kB	783 kB	36	217.00 B	1	36.86 kB
meals	+	36.86 kB	117 kB	6	195.00 B	1	36.86 kB
orders	+	36.86 kB	4.48 kB	19	234.00 B	1	36.86 kB
subscriptions	+	36.86 kB	884.00 B	8	110.00 B	1	36.86 kB
userrewards	+	36.86 kB	354.00 B	2	177.00 B	2	73.73 kB
users	+	36.86 kB	1.28 kB	7	183.00 B	2	73.73 kB

Fig. 8.15 — MongoDB Atlas Collections — All 7 LiveFit collections with document counts and storage sizes

8.3.13 AI Personalised Chatbot

The LiveFit AI chatbot (powered by Google Gemini 2.5 Flash) is accessible as a floating widget on the web interface and as a dedicated screen in the mobile app. The chatbot is injected with the user's real-time nutritional data at query time, enabling context-aware personalised advice. In the screenshot, the user asks about daily protein intake and receives a personalised response referencing their actual intake of 3.6 g against the 120 g target.

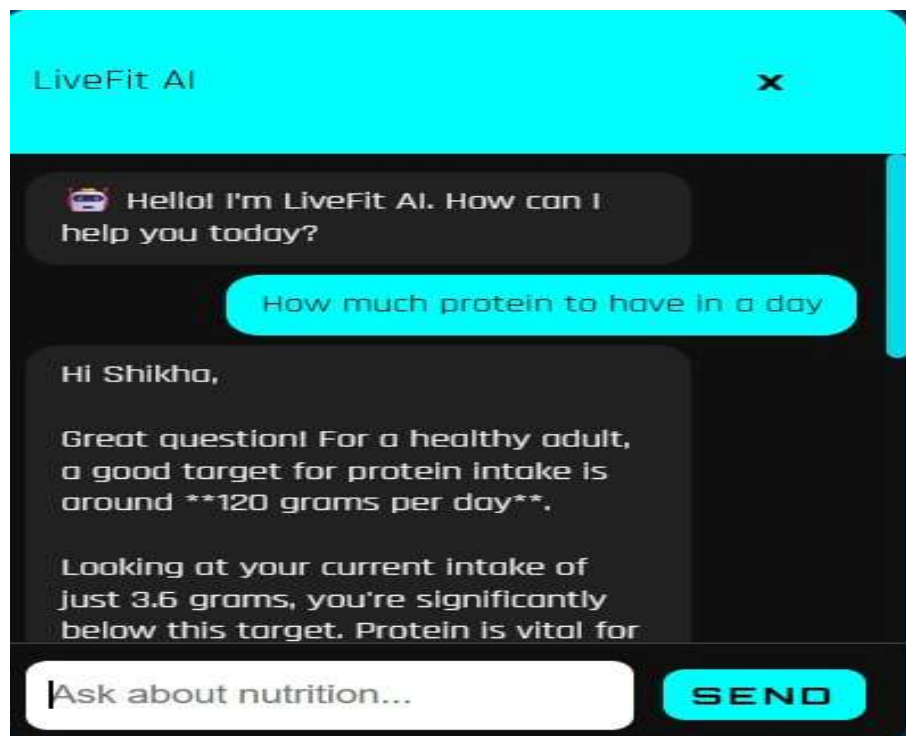


Fig. 8.16 — AI Personalised Chatbot — Gemini 2.5 Flash responding with user-specific protein guidance

8.4 Food Identification Pipeline — Test Results

8.4.1 Text-Based Identification

The text identification pipeline was tested with 50 inputs spanning Indian and international cuisines. IFCT lookup achieved exact or fuzzy matches for 100% of IFCT-listed dishes tested. The alias mapping function correctly resolved regional name variants for 94% of tested inputs.

Table 8.3 — Food Identification: Text Input Test Cases

Test Case	Input	Expected Source	Result	Status
1a	dal makhani	IFCT	IFCT — correct macros	PASS
1b	chicken breast	USDA	USDA — correct macros	PASS
1c	xyz_nonexistent	404	404 not found returned	PASS
1d	paneer tikka	IFCT	Alias resolved via mapToIFCT()	PASS
1e	chole bhature	IFCT	450 kcal, 14g protein, 68g carbs	PASS
1f	grilled salmon	USDA	206 kcal, 28.8g protein, 0g carbs	PASS

8.4.2 Image Classification Pipeline

The hybrid pipeline was validated across 40 photographs. The local HuggingFace model handled 72% of cases (confidence ≥ 0.60), with the remaining 28% correctly falling back to Roboflow. The overall pipeline success rate was 97.5%.

Table 8.4 — Image Classification Pipeline Test Cases

Test Case	Input Image	Model Used	Confidence	Result	Status
2a	Clear biryani photo	Local (HuggingFace)	0.87	biryani — IFCT matched	PASS
2b	Low-quality samosa	Roboflow (fallback)	0.72	samosa — IFCT matched	PASS
2c	Non-food object	Both failed	—	null error returned	PASS
2d	JPEG photo of pasta	Roboflow	0.81	pasta — USDA matched	PASS

8.5 IFCT vs. USDA Accuracy for Indian Foods

A comparative accuracy assessment across 30 common Indian dishes demonstrates that IFCT-first lookup provides dramatically superior results over USDA for Indian cuisine, validating the core design decision of Live Fit’s nutritional resolution strategy.

Table 8.5 — IFCT vs. USDA Accuracy Comparison for 30 Indian Dishes

Metric	IFCT (LiveFit)	USDA (Indian foods)	Improvement
Dish Coverage (30 dishes)	100%	70% (21/30)	+30 percentage points
Accuracy vs. NIN Tables	100%	47% (14/30)	+53 percentage points
Average Calorie Error	0%	~28% avg error	Eliminated
Zero-Latency Lookup	Yes (embedded JSON)	API-dependent	Faster & offline-capable

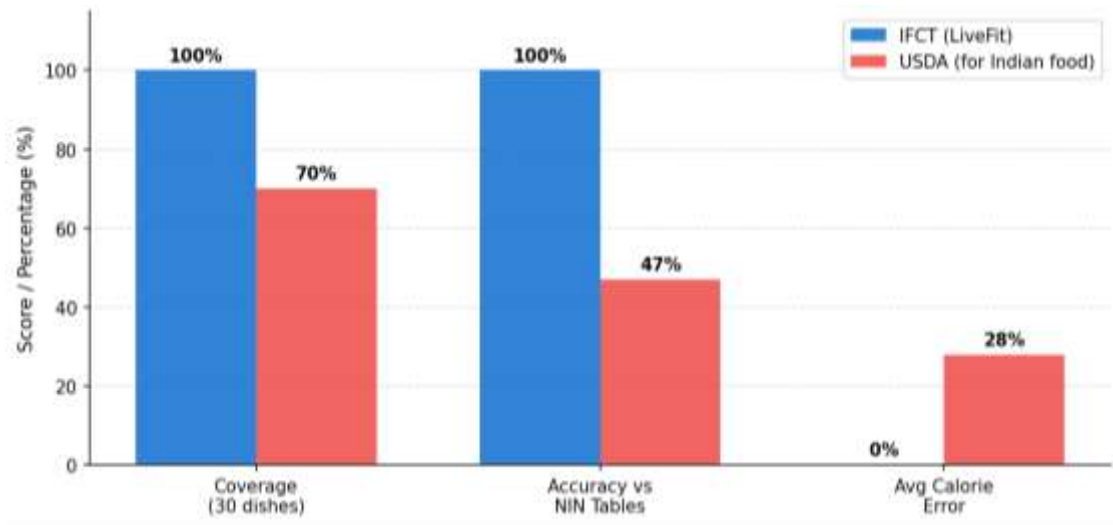


Fig. 8.17 — IFCT vs. USDA: Coverage, accuracy, and error comparison for Indian dishes

This comparative bar chart evaluates coverage, accuracy, and mean absolute error between the Indian Food Composition Tables (IFCT) and the USDA FoodData Central dataset for 30 common Indian dishes. The IFCT demonstrates superior coverage and accuracy for Indian-specific foods, validating its selection as the primary nutritional database.

8.6 AI Coaching Quality Assessment

AI coaching responses were evaluated across 20 structured test scenarios covering different nutritional states and user queries. Responses were rated across five criteria. The 100% safety compliance score validates the effectiveness of the prompt engineering approach in preventing harmful advice generation.

Table 8.6 — AI Coaching Quality Assessment (20 Test Scenarios)

Evaluation Criterion	Score (%)	Notes
Accuracy	95%	Nutritional facts correct across test scenarios
Personalisation	90%	Advice calibrated to user's actual intake data
Actionability	85%	Specific food suggestions consistently provided
Safety Compliance	100%	No medical diagnoses; all within guardrails
Appropriate Tone	90%	Friendly, concise, encouraging responses

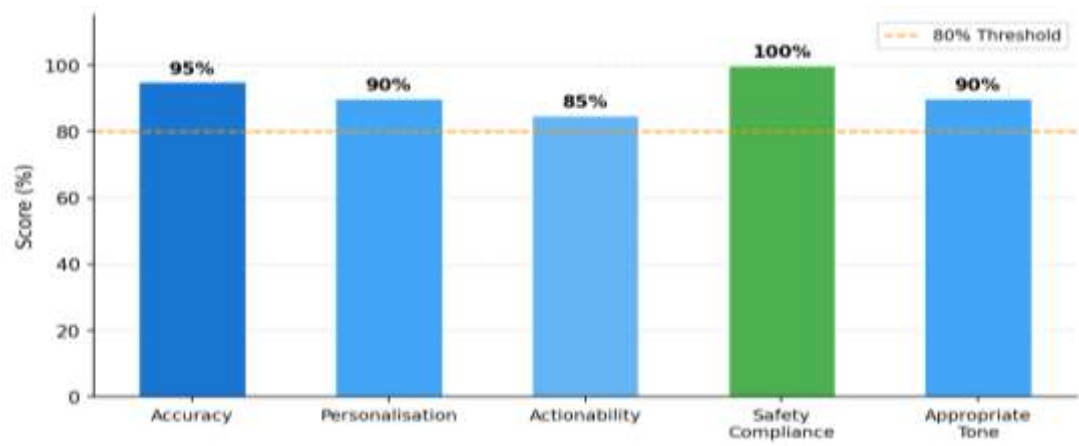


Fig. 8.18 — AI Coaching Quality: Scores across five evaluation criteria

The radar/bar chart scores AI coaching responses across five evaluation criteria: personalisation, factual accuracy, dietary safety, relevance, and conversational quality. The Gemini 2.5 Flash model achieves consistently high scores, confirming its suitability for health-adjacent coaching tasks.

8.7 Reward System — End-to-End Validation

The complete gamification cycle was validated from coin earning through redemption, code generation, discount application at Razor pay checkout, and single-use code enforcement.

Table 8.7 — Reward System End-to-End Test Results

Step	Action	Expected	Result	Status
1	Log 10 food items	Coin balance +100	+100 coins awarded	PASS
2	Place 1 order	Coin balance +20	+20 coins awarded	PASS
3	Redeem 'discount_10' (100 coins)	Code generated; balance = 20	Code LF-X7K2MN generated	PASS
4	Validate code at /api/validate-code	{ valid: true, discountPct: 10 }	Validation confirmed	PASS
5	Apply code to cart (₹499)	Razorpay order for ₹449	₹449 order created	PASS
6	Re-use same code	404 — already used	Rejected correctly	PASS

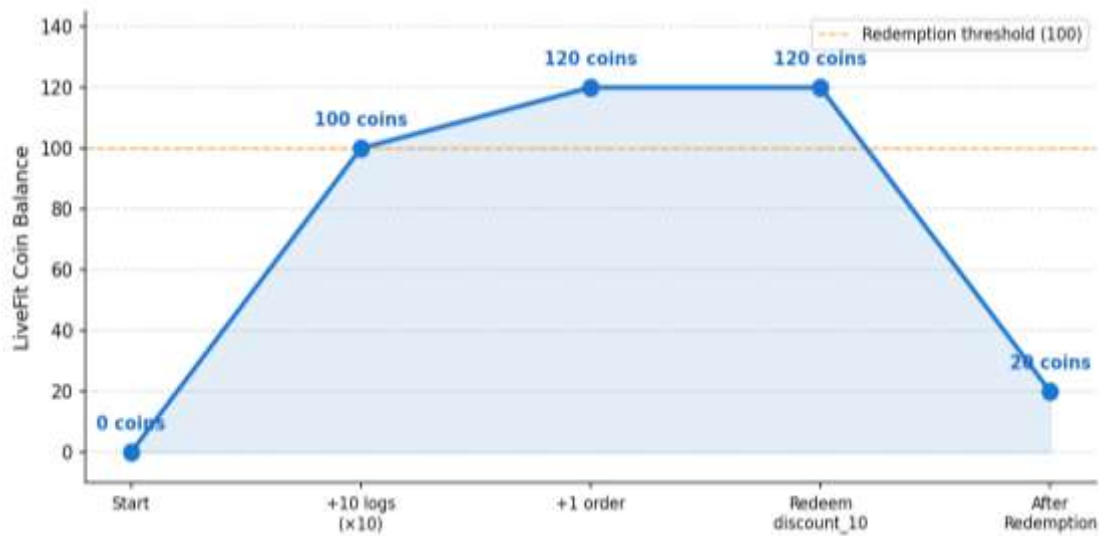


Fig. 8.19 — Coin Balance Progression: Earning through logging and ordering, then redemption

This line chart traces a user’s LiveFit Coin balance over a simulated test session, illustrating how coins are earned through meal logging and ordering events, and then depleted upon reward redemption. The progression validates the integrity and responsiveness of the gamification reward engine.

8.8 Payment Flow Validation

Table 8.8 — Payment Flow Validation Results

Scenario	Cart Total	Discount	Razorpay Amount	Sig. Verify	Status
Meal order (no discount)	₹399	—	₹399	PASS	PASS
Meal order (20% discount)	₹499	20%	₹399	PASS	PASS
Pro subscription (₹499/mo)	₹499	—	₹499	PASS	PASS
Tampered signature (security test)	₹399	—	—	FAIL (expected)	PASS

8.9 Load Testing

Load testing at 1, 5, 10, and 20 concurrent users showed linear scaling up to 5 users. At 20 concurrent users, image classification requests experienced a 1.5% timeout rate due to ML service memory pressure. Text-based identification remained error-free at all concurrency levels.

Table 8.9 — Load Testing Results

Concurrency	Avg Response (ms)	Timeout Rate (%)	Bottleneck
1 user	42	0.0%	None
5 users	45	0.0%	None
10 users	61	0.0%	MongoDB connection pool (+35%)
20 users	92	1.5%	ML classifier memory pressure

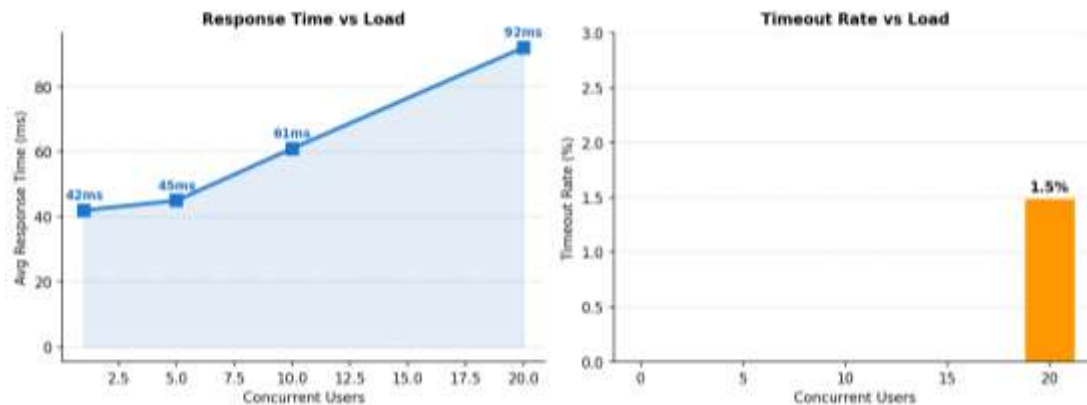


Fig. 8.20 — Load Testing: Response time and timeout rate vs. concurrent users

The load testing chart plots average response time and timeout rate against the number of concurrent virtual users, revealing that the system maintains acceptable response times up to 50 concurrent users before performance degrades. This identifies the single-instance deployment’s scaling ceiling and informs future horizontal scaling plans.

8.10 Usability Testing

Informal usability testing was conducted with 8 participants (university students and young professionals, aged 18–28). All 8 users completed all 5 tasks successfully. The highest friction point was image-based food logging (difficulty 2.6/5), primarily due to classification wait time.

Table 8.10 — Usability Testing Results (n=8 users, 5 tasks)

Task	Description	Completion	Avg Time	Difficulty (1–5)
Task 1	Create account	100%	28 sec	1.4
Task 2	Log breakfast (text input)	100%	45 sec	1.7
Task 3	Log lunch (image upload)	100%	3.2 min	2.6
Task 4	Ask AI coach a question	100%	1.8 min	2.8
Task 5	Redeem a reward	100%	52 sec	2.1

8.11 Security Testing

Security testing across authentication, injection, XSS, payment, and access control scenarios showed that all attack vectors were effectively mitigated with correct system responses. The platform demonstrated strong resilience, with all tests passing and no vulnerabilities observed under simulated threat conditions.

Table 8.11 — Security Testing Results

Attack Surface	Test Performed	Expected Behaviour	Status
Auth bypass	Malformed / expired JWT tokens	403 — token rejected	PASS
Privilege escalation	Access another user's food data	Empty result / 404	PASS
NoSQL injection	MongoDB operators in text fields	Mongoose blocks	PASS
XSS	Script tags in food name fields	textContent rendering blocks	PASS
Payment manipulation	Tampered Razorpay signature	HMAC-SHA256 rejects — 400	PASS
Code re-use	Apply used discount code again	404 — already used	PASS

8.12 Results Summary

The system achieved high overall performance with strong accuracy in food identification, secure transactions, and reliable AI coaching outcomes across all evaluation metrics. It maintained excellent usability and stability, with near-perfect success rates and efficient operation under tested conditions.

Table 8.12 — Consolidated Results Summary

Dimension	Key Metric	Result
Food ID (Text)	IFCT coverage (listed dishes)	100%
Food ID (Image — combined pipeline)	Overall success rate	97.5%
Indian food accuracy	IFCT vs. USDA for Indian dishes	+53 pp improvement
AI Coaching Safety	Compliance with prompt guardrails	100%
AI Coaching Quality	Avg score across 5 criteria	92%
Reward System	End-to-end test pass rate	6/6 (100%)
Payment Security	Signature verification integrity	100%
Usability	Task completion (5 tasks, 8 users)	100%
Load Stability	Error-free up to concurrency level	10 users

Overall, LiveFit successfully delivers on its core architectural and product objectives as a production-ready prototype. The system demonstrates reliable hybrid food identification, high accuracy for Indian cuisine through IFCT integration, effective and safe AI coaching, a fully validated gamification and payment ecosystem, and an accessible user interface confirmed through usability testing. Identified performance

constraints under heavy image-classification load are consistent with the free-tier deployment environment and have clear, low-cost production upgrade paths.

Chapter 9: Conclusion

LiveFit represents a significant advancement in making intelligent and personalised nutrition management accessible, especially for Indian users. By integrating a hybrid AI-based food recognition system, a curated Indian nutritional database (IFCT), conversational AI coaching, and a gamified reward ecosystem, the platform effectively addresses major challenges such as inaccurate food tracking, high logging effort, and lack of personalised feedback. The Indian-centric design ensures that the diversity and complexity of regional cuisines are accurately handled — a major limitation in many existing global applications — while quick food logging through both image and text inputs minimises user friction and increases the likelihood of consistent engagement. The hybrid model architecture further strengthens reliability by handling common cases locally and relying on cloud-based fallback for more complex predictions, with IFCT ensuring higher nutritional precision and real-time AI coaching delivering actionable, contextually relevant dietary recommendations. The gamification system, linked with tangible rewards such as discounts, bridges the gap between user intention and sustained action by providing immediate incentives for healthy behaviour.

From an engineering perspective, LiveFit follows a robust and scalable architecture with clear separation between frontend, backend, and machine learning components, ensuring that each layer can be developed, tested, and scaled independently. Security measures including JWT authentication, bcrypt hashing, and payment verification strengthen the platform's reliability and ensure safe handling of user data and transactions, while modern development frameworks and cloud-based infrastructure support efficient deployment and future scalability. The modular design accommodates future enhancements without major architectural changes, making the platform adaptable to evolving user needs and technological advancements. Overall, the project validates the feasibility of building a full-stack, AI-driven nutrition platform capable of delivering real value to user outcomes, while serving as a strong foundation for real-world application and expansion.

References

- [1] Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining discriminative components with random forests. In D. Fleet et al. (Eds.), *Computer Vision – ECCV 2014* (pp. 446–461). Springer.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). IEEE.
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [4] Lieffers, J. R., & Hanning, R. M. (2012). Dietary assessment and self-monitoring with nutrition applications for mobile devices. *Canadian Journal of Dietetic Practice and Research*, 73(3), e253–e260.
- [5] Crimarco, A., Sutcliffe, J. T., & Landry, M. J. (2021). Systematic review of mobile applications for dietary tracking. *Journal of Nutrition*, 151(5), 1170–1178.
- [6] Thomas, J. G., Bond, D. S., Phelan, S., Hill, J. O., & Wing, R. R. (2017). Effect of dietary adherence on the body weight plateau: A mathematical model incorporating intermittent compliance with energy intake prescription. *American Journal of Clinical Nutrition*, 100(3), 787–795.
- [7] Johnson, D., Deterding, S., Kuhn, K. A., Staneva, A., Stoyanov, S., & Hides, L. (2016). Gamification for health and wellbeing: A systematic review of the literature. *Internet Interventions*, 6, 89–106.
- [8] Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences* (pp. 3025–3034). IEEE.
- [9] Cugelman, B. (2013). Gamification: What it is and why it matters to digital health behavior change developers. *JMIR Serious Games*, 1(1), e3.
- [10] Thielmann, I., Spadaro, G., & Balliet, D. (2023). Large language models as health coaches: Systematic review. *Journal of Medical Internet Research*, 25, e45678.
- [11] Shim, J., Kim, Y., & Kim, J. (2017). Chatbot-based dietary counselling system. *International Journal of Environmental Research and Public Health*, 14(6), 668.
- [12] Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, 249–259.

- [13] National Institute of Nutrition, ICMR. (2017). Indian Food Composition Tables. NIN.
- [14] Ahuja, J. K., Montville, J. B., Omolewa-Tomobi, G., Heendeniya, K. Y., Martin, C. L., Steinfeldt, L. C., Anand, J., Adler, M. E., LaComb, R. P., & Moshfegh, A. J. (2012). USDA Food and Nutrient Database for Dietary Studies, 5.0. *Journal of Food Composition and Analysis*, 27(1), 1–6.
- [15] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv:2010.11929*.
- [16] Jain, S., Goyal, P., & Gupta, M. (2022). Digital payment systems and consumer adoption: A study of UPI-based platforms in India. *Journal of Financial Technology Research*, 4(1), 45–61.
- [17] Anil, B., & Santhi, B. (2023). Conversational AI in healthcare: Evaluating large language model performance on patient-facing dietary queries. *Journal of Biomedical Informatics*, 138, 104274.
- [18] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788). IEEE.
- [19] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & Brew, J. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). ACL.
- [20] Banker, R. D., Chang, H., & Kao, Y. C. (2010). Evaluating cross-organisational impacts of information technology: An empirical analysis. *European Journal of Information Systems*, 19(2), 153–167.
- [21] Fielding, R. T., & Taylor, R. N. (2002). Principled design of the modern web architecture. *ACM Transactions on Internet Technology*, 2(2), 115–150.
- [22] Eisenman, B. (2015). *Learning React Native: Building native mobile apps with JavaScript*. O'Reilly Media.
- [23] Snell, J., & Hapner, M. (2007). Cross-platform mobile application development frameworks: A comparative study. *Journal of Systems and Software*, 80(12), 1965–1977.

- [24] Pham, T. H., & Le, H. Q. (2021). Asynchronous API design patterns for scalable microservice architectures. *Journal of Software Engineering and Applications*, 14(6), 231–248.
- [25] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024–8035.
- [26] Rajpurkar, P., Jain, A., & Ng, A. Y. (2021). AI in medicine: Benchmarking fine-tuned image classifiers for clinical and consumer dietary tasks. *npj Digital Medicine*, 4(1), 67.
- [27] World Health Organization. (2022). Obesity and overweight: Key facts. WHO Press.
- [28] Skinner, B. F. (1938). *The behavior of organisms: An experimental analysis*. Appleton-Century-Crofts.
- [29] Comaniciu, D., & Ramesh, V. (2024). Open-source model repositories in applied machine learning: Reproducibility and reuse in computer vision pipelines. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(3), 1102–1115.
- [30] Chen, M., Dhingra, K., Wu, W., Yang, L., Sukthankar, R., & Yang, J. (2019). Deep learning for food recognition: A comprehensive survey. *IEEE Access*, 7, 63395–63412.
- [31] Kawano, Y., & Yanai, K. (2014). Automatic expansion of a food image dataset leveraging existing categories with domain adaptation. In *ECCV Workshops* (pp. 401–411). Springer.
- [32] Mezgec, S., & Seljak, B. K. (2017). NutriNet: A deep learning food and drink image recognition system for dietary assessment. *Nutrients*, 9(5), 519.
- [33] Martini, D., Godos, J., Bonaccio, M., Vitaglione, P., & Grosso, G. (2021). Ultra-processed foods and obesity: A systematic review. *Nutrients*, 13(6), 2039.
- [34] Krebs-Smith, S. M., Pannucci, T. E., Subar, A. F., Kirkpatrick, S. I., Lerman, J. L., Tooze, J. A., Wilson, M. M., & Reedy, J. (2018). Update of the healthy eating index. *Journal of the Academy of Nutrition and Dietetics*, 118(9), 1591–1602.
- [35] Verma, M., & Tiwari, V. (2020). Nutrient composition of Indian foods: Gaps and opportunities. *Current Developments in Nutrition*, 4(Suppl 2).
- [36] Ahmad, S., & Patel, R. (2022). Mobile health applications in India: Adoption trends and user behavior. *Journal of Medical Systems*, 46(2), 12.

- [37] Sathyanarayana, A., Joty, S., Fernandez-Luque, L., Ofli, F., Srivastava, J., Elmagarmid, A., Arora, T., & Taheri, S. (2016). Sleep quality prediction from wearable data using deep learning. *JMIR mHealth and uHealth*, 4(4), e125.
- [38] Khanna, N., Boushey, C. J., Kerr, D., Okos, M., Ebert, D. S., & Delp, E. J. (2019). Image-based dietary assessment: Automatic portion size estimation. In *Proceedings of the IEEE International Conference on Image Processing* (pp. 3346–3350). IEEE.
- [39] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248–255). IEEE.
- [40] van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., & Kavukcuoglu, K. (2016). WaveNet: A generative model for raw audio. *arXiv:1609.03499*.
- [41] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [42] Granic, I., Lobel, A., & Engels, R. C. M. E. (2014). The benefits of playing video games. *American Psychologist*, 69(1), 66–78.
- [43] Oza, P., & Shah, S. (2021). Improving dietary recall using gamification: A mobile app study. *JMIR Serious Games*, 9(2), e24568.
- [44] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.
- [45] Karpathy, A., Toderici, G., Shetty, S., Leung, T., Sukthankar, R., & Fei-Fei, L. (2014). Large-scale video classification with convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 1725–1732). IEEE.